

## SOLIDWORKS API (NET VBA C#) — Examples (26/50): Get\_Core\_Feature\_Example\_VBNET Get\_DimXpert\_Display\_Dimensions\_and\_Feat

SOLIDWORKS API Documentation (converted from CHM)

## Get Core Feature Data Example (VB.NET)

This example shows how to get the data for a core feature.

```

-----
Preconditions:
1. Open a model document that contains a core feature.
2. Open the Immediate window.
3. Select the core feature in the FeatureManager design tree.
-----
Postconditions:
1. Prints the core feature data to the Immediate window.
2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swCoreFeat As CoreFeatureData
        Dim b As Boolean
        Dim nam As String
        Dim cap As Boolean
        Dim d1 As Double
        Dim d2 As Double
        Dim ang As Double
        Dim useDr As Boolean
        Dim Drout As Boolean
        Dim rev As Boolean
        Dim typ1 As Integer
        Dim typ2 As Integer
        Dim dir1 As Object = Nothing
        Dim dir2 As Object = Nothing
        Dim e1 As Integer
        Dim e2 As Integer
        Dim ct As Integer

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swCoreFeat = swFeat.GetDefinition

        'Roll back to the feature before the core feature
        b = swCoreFeat.AccessSelections(swModel, Nothing)

        'Get the bounding sketch of the core feature
        swFeat = swCoreFeat.BoundingSketch
        nam = swFeat.Name
        Debug.Print("Name of bounding sketch = " & nam)
        cap = swCoreFeat.CapEnds
        Debug.Print("Are ends capped? " & cap)
        d1 = swCoreFeat.Depth(0)
        Debug.Print("Depth along extraction direction = " & d1)
        d2 = swCoreFeat.Depth(1)
        Debug.Print("Depth away from extraction direction = " & d2)
        useDr = swCoreFeat.UseDraft
        Debug.Print("Drafted? " & useDr)
        If useDr Then
            ang = swCoreFeat.DraftAngle
            Debug.Print("Angle of draft = " & ang)
            Drout = swCoreFeat.DraftOutward
            Debug.Print("Drafted outward? " & Drout)
        End If
        e1 = swCoreFeat.EndCondition(0)
        Debug.Print("End condition along extraction = " & e1)
        e2 = swCoreFeat.EndCondition(1)
        Debug.Print("End condition away from extraction = " & e2)
        rev = swCoreFeat.ReverseDirection
        Debug.Print("Direction of extraction reversed? " & rev)
        ct = swCoreFeat.GetExtractionDirection(typ1, dir1, typ2, dir2)
        Debug.Print("Number of entities that define extraction = " & ct)

        'Roll forward
        swCoreFeat.ReleaseSelectionAccess()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

## Get Corner Points of a Reference Plane Example (C#)

This example shows how to obtain the four corner points of a reference plane.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Creates 3DSketch1 containing four corner points of the reference plane.
// 3. Gets the coordinates of each corner point.
// 4. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        bool boolstatus;
        Feature swFeature;
        RefPlane swRefPlane;
        ModelDocExtension swModelExt;
        SelectionMgr swSelMgr;
        Double[] vArrayData;
        Object[] cpObj;
        MathPoint[] vMathPoints = new MathPoint[4];
        int i;
        SketchManager sketchMgr;
        SketchPoint sketchPt;
        string filename;
        int errors;
        int warnings;
        public void Main()
        {
            filename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\swutilities\\bracket_a.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(filename, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref
errors, ref warnings);
            swModelExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            sketchMgr = swModel.SketchManager;
            boolstatus = swModelExt.SelectByID2("Plane4", "PLANE", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swRefPlane = (RefPlane)swFeature.GetSpecificFeature2();
            cpObj = (Object[])swRefPlane.CornerPoints;

            //Four (4) MathPoint objects are always returned
            sketchMgr.Insert3DSketch(true);
            for (i = 0; i <= cpObj.GetUpperBound(0); i++)
            {
                vMathPoints[i] = (MathPoint)cpObj[i];
                vArrayData = (Double[])(vMathPoints[i].ArrayData);
                Debug.Print(" Point x = " + vArrayData[0]);
                Debug.Print(" Point y = " + vArrayData[1]);
                Debug.Print(" Point z = " + vArrayData[2]);
                Debug.Print("");
                sketchPt = sketchMgr.CreatePoint(vArrayData[0], vArrayData[1], vArrayData[2]);
            }
            sketchMgr.Insert3DSketch(true);
        }
        public SldWorks swApp;
    }
}
```

## Get Corner Points of a Reference Plane Example (VBA)

This example shows how to obtain the four corner points of a reference plane.

```
' -----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Creates 3DSketch1 containing four corner points of the reference plane.
' 3. Gets the coordinates of each corner point.
' 4. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim swFeature As SldWorks.Feature
Dim swRefPlane As SldWorks.RefPlane
Dim swModelExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim vMathPoints As Variant
Dim vArrayData As Variant
Dim pMathPoint As SldWorks.MathPoint
Dim i As Integer
Dim swSketch As SldWorks.Sketch
Dim sketchMgr As SldWorks.SketchManager
Dim sketchPt As SldWorks.SketchPoint
Dim swRefPlaneFeatData As SldWorks.RefPlaneFeatureData
Dim filename As String
Dim errors As swFileLoadError_e
Dim warnings As swFileLoadWarning_e
```

```
Sub main()
    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\swutilities\bracket_a.sldprt"
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6(filename, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    Set sketchMgr = swModel.SketchManager
    boolstatus = swModelExt.SelectByID2("Plane4", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set swFeature = swSelMgr.GetSelectedObject5(1)
    Set swRefPlane = swFeature.GetSpecificFeature2

    vMathPoints = swRefPlane.CornerPoints 'Four (4) MathPoint objects are always returned
    sketchMgr.Insert3DSketch True
    For i = 0 To UBound(vMathPoints)
        vArrayData = vMathPoints(i).ArrayData
        Debug.Print " Point x = " & vArrayData(0)
        Debug.Print " Point y = " & vArrayData(1)
        Debug.Print " Point z = " & vArrayData(2)
        Debug.Print
        Set sketchPt = sketchMgr.CreatePoint(vArrayData(0), vArrayData(1), vArrayData(2))
    Next i
    sketchMgr.Insert3DSketch True
End Sub
```

## Get Corner Points of a Reference Plane Example (VB.NET)

This example shows how to obtain the four corner points of a reference plane.

```
'-----  
' Preconditions:  
' 1. Verify that the part to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the part.  
' 2. Creates 3DSketch1 containing four corner points of the reference plane.  
' 3. Gets the coordinates of each corner point.  
' 4. Examine the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
Dim swModel As ModelDoc2  
Dim boolstatus As Boolean  
Dim swFeature As Feature  
Dim swRefPlane As RefPlane  
Dim swModelExt As ModelDocExtension  
Dim swSelMgr As SelectionMgr  
Dim vMathPoints As Object  
Dim vArrayData As Object  
Dim pMathPoint As MathPoint  
Dim i As Integer  
Dim swSketch As Sketch  
Dim sketchMgr As SketchManager  
Dim sketchPt As SketchPoint  
Dim swRefPlaneFeatData As RefPlaneFeatureData  
Dim filename As String  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
  
    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\swutilities\bracket_a.sldprt"  
    swModel = swApp.OpenDoc6(filename, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    swModelExt = swModel.Extension  
    swSelMgr = swModel.SelectionManager  
    sketchMgr = swModel.SketchManager  
    boolstatus = swModelExt.SelectByID2("Plane4", "PLANE", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)  
    swFeature = swSelMgr.GetSelectedObject6(1, -1)  
    swRefPlane = swFeature.GetSpecificFeature2  
    vMathPoints = swRefPlane.CornerPoints 'Four (4) MathPoint objects are always returned  
    sketchMgr.Insert3DSketch(True)  
    For i = 0 To UBound(vMathPoints)  
        vArrayData = vMathPoints(i).ArrayData  
        Debug.Print(" Point x = " & vArrayData(0))  
        Debug.Print(" Point y = " & vArrayData(1))  
        Debug.Print(" Point z = " & vArrayData(2))  
        Debug.Print("")  
        sketchPt = sketchMgr.CreatePoint(vArrayData(0), vArrayData(1), vArrayData(2))  
    Next i  
    sketchMgr.Insert3DSketch(True)  
End Sub  
  
Public swApp As SldWorks  
End Class
```

## Get Corresponding Entities Between Parts and Drawing Views Example (C#)

This example shows how to get corresponding entities or objects between a part and its drawing.

```
//-----
// Preconditions:
// 1. Ensure that the specified part and drawing exist.
// 2. Open the Immediate window.
// 3. Uncomment the subroutine you want to run in Main().
// 4. At the pause, select a face, edge, vertex, feature, annotation,
//    or sketch segment.
// 5. Press F5.
//
// Postconditions:
// 1. Inspect the Immediate window.
// 2. If a corresponding face, edge, or vertex is found, it is selected in the
//    underlying part or drawing.
//
// NOTE: Because the models are used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;

namespace GetCorresponding_CSharp
{
    public partial class SolidWorksMacro
    {
        DocumentSpecification docSpec;
        ModelDoc2 swModelPart;
        ModelDoc2 swModelDrawing;
        DrawingDoc swDrawing;
        View swView;
        int lErr = 0;
        SelectionMgr selMgr;
        Entity inputEntity;
        Entity outputEntity;
        bool bSelected;
        object inputObject;
        object outputObject;
        DrawingComponent drComp;

        public void Main()
        {
            //Uncomment the subroutine you want to run; comment the other one
            //ViewToPart()
            PartToView();
        }

        public void ViewToPart()
        {
            docSpec = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\sample
            swModelPart = swApp.OpenDoc7(docSpec);

            docSpec = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\sample
            swModelDrawing = swApp.OpenDoc7(docSpec);

            swDrawing = (DrawingDoc)swModelDrawing;
            swView = (View)((Feature)(swDrawing.FeatureByName("Drawing View1"))).GetSpecificFeature();

            swApp.ActivateDoc3(swModelPart.GetTitle(), true, (int)swRebuildOnActivation_e.swUserDecision, lErr);

            selMgr = (SelectionMgr)swModelPart.SelectionManager;
            swModelPart.ClearSelection2(true);

            System.Diagnostics.Debugger.Break();
            // Select something in the model and press F5

            switch (selMgr.GetSelectedObjectType3(1, -1))
            {
                case (int)swSelectType_e.swSelFACES:
                case (int)swSelectType_e.swSelEDGES:
                case (int)swSelectType_e.swSelVERTICES:

                    inputEntity = (Entity)selMgr.GetSelectedObject6(1, -1);

                    Debug.Print("Using IView::GetCorrespondingEntity()");

                    outputEntity = (Entity)swView.GetCorrespondingEntity(inputEntity);

                    if (outputEntity == null)
                    {
                        Debug.Print("No corresponding entity found in the drawing view");
                    }
                    else
                    {
                        Debug.Print("Corresponding entity found....selecting in drawing");
                    }
            }
        }
    }
}
```

```

        swApp.ActivateDoc3(swModelDrawing.GetTitle(), false, (int)swRebuildOnActivation_e.swDontRebuildActiveDoc, lErr);
        bSelected = outputEntity.Select4(false, null);
    }

    break;
case (int)swSelectType_e.swSelNOTHING:
    break;
default:
    inputObject = selMgr.GetSelectedObject6(1, -1);

    Debug.Print("Using IView::GetCorresponding()");

    outputObject = swView.GetCorresponding(inputObject);

    if (outputObject == null)
    {
        Debug.Print("No corresponding object found in the drawing view");
    }
    else
    {
        Debug.Print("Corresponding object found in the drawing view");
    }
    break;
}

}

}

public void PartToView()
{
    docSpec = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\sample
    swModelPart = swApp.OpenDoc7(docSpec);

    docSpec = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\sample
    swModelDrawing = swApp.OpenDoc7(docSpec);

    swDrawing = (DrawingDoc)swModelDrawing;

    swApp.ActivateDoc3(swModelDrawing.GetTitle(), false, (int)swRebuildOnActivation_e.swDontRebuildActiveDoc, lErr);

    selMgr = (SelectionMgr)swModelDrawing.SelectionManager;
    swModelDrawing.ClearSelection2(true);

    System.Diagnostics.Debugger.Break();
    // Select something in the drawing and press F5

    swView = (View)((Feature)(swDrawing.FeatureByName("Drawing View1"))).GetSpecificFeature();

    switch (selMgr.GetSelectedObjectType3(2, -1))
    {
        case (int)swSelectType_e.swSelFACES:
        case (int)swSelectType_e.swSelEDGES:
        case (int)swSelectType_e.swSelVERTICES:

            inputEntity = (Entity)selMgr.GetSelectedObject6(2, -1);
            drComp = (DrawingComponent)selMgr.GetSelectedObjectsComponent4(2, -1);

            Debug.Print("Using IModelDocExtension::GetCorrespondingEntity2()");

            outputEntity = (Entity)swModelPart.Extension.GetCorrespondingEntity2(inputEntity);

            if (outputEntity == null)
            {
                Debug.Print("No corresponding entity found in the part");
            }
            else
            {
                Debug.Print("Corresponding entity found...selecting in part");
                swApp.ActivateDoc3(swModelPart.GetTitle(), false, (int)swRebuildOnActivation_e.swDontRebuildActiveDoc, lErr);
                bSelected = outputEntity.Select4(false, null);
            }

            break;
        case (int)swSelectType_e.swSelNOTHING:
            break;
        default:
            inputObject = selMgr.GetSelectedObject6(2, -1);
            drComp = (DrawingComponent)selMgr.GetSelectedObjectsComponent4(2, -1);

            Debug.Print("Using IModelDocExtension::GetCorresponding2()");

            outputObject = swModelPart.Extension.GetCorresponding2(inputObject);

            if (outputObject == null)
            {
                Debug.Print("No corresponding object found in the part");
            }
            else
            {
                Debug.Print("Corresponding object found in the part");
            }
            break;
    }
}

```

```
        }  
    }  
    // The SldWorks swApp variable is pre-assigned for you.  
    public SldWorks swApp;  
}  
}
```



## Get Corresponding Entities Between Parts and Drawing Views Example (VBA)

This example shows how to get corresponding entities or objects between a part and its drawing.

```

'-----
' Preconditions:
' 1. Ensure that the specified part and drawing exist.
' 2. Open the Immediate window.
' 3. Run either subroutine.
' 4. At the pause, select a face, edge, vertex, feature, annotation,
'    or sketch segment.
' 5. Press F5.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. If a corresponding face, edge, or vertex is found, it is selected in the
'    underlying part or drawing.
'
' NOTE: Because the models are used elsewhere, do not save changes.
'-----
Option Explicit
Dim swApp As SldWorks.SldWorks
Dim docSpec As SldWorks.DocumentSpecification
Dim swModelPart As SldWorks.ModelDoc2
Dim swModelDrawing As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim lErr As Long
Dim selMgr As SldWorks.SelectionMgr
Dim inputEntity As SldWorks.Entity
Dim outputEntity As SldWorks.Entity
Dim bSelected As Boolean
Dim inputObject As Object
Dim outputObject As Object
Dim drComp As SldWorks.DrawingComponent

Sub PartToView()

    Set swApp = Application.SldWorks

    Set docSpec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.SLDPRPT")
    Set swModelPart = swApp.OpenDoc7(docSpec)

    Set docSpec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.SLDDRW")
    Set swModelDrawing = swApp.OpenDoc7(docSpec)

    Set swDrawing = swModelDrawing
    Set swView = swDrawing.FeatureByName("Drawing View1").GetSpecificFeature()

    swApp.ActivateDoc3 swModelPart.GetTitle, True, swRebuildOnActivation_e.swUserDecision, lErr

    Set selMgr = swModelPart.SelectionManager
    swModelPart.ClearSelection2 True

    Debug.Assert False ' Select something in the model and press F5

    Select Case selMgr.GetSelectedObjectType3(1, -1)
    Case swSelFACES, swSelEDGES, swSelVERTICES

        Set inputEntity = selMgr.GetSelectedObject6(1, -1)

        Debug.Print "Using IView::GetCorrespondingEntity()"

        Set outputEntity = swView.GetCorrespondingEntity(inputEntity)

        If outputEntity Is Nothing Then
            Debug.Print "No corresponding entity found in the drawing view"
        Else
            Debug.Print "Corresponding entity found...selecting in drawing"
            swApp.ActivateDoc3 swModelDrawing.GetTitle, False, swRebuildOnActivation_e.swDontRebuildActiveDoc, lErr
            bSelected = outputEntity.Select4(False, Nothing)
        End If

    Case swSelNOTHING

    Case Else

        Set inputObject = selMgr.GetSelectedObject6(1, -1)

        Debug.Print "Using IView::GetCorresponding()"
    End Select
End Sub

```

```

    Set outputObject = swView.GetCorresponding(inputObject)

    If outputObject Is Nothing Then
        Debug.Print "No corresponding object found in the drawing view"
    Else
        Debug.Print "Corresponding object found in the drawing view"
    End If
End Select
End Sub

Sub ViewToPart()
    Set swApp = Application.SldWorks

    Set docSpec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.SLDPRT")
    Set swModelPart = swApp.OpenDoc7(docSpec)

    Set docSpec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.SLDDRW")
    Set swModelDrawing = swApp.OpenDoc7(docSpec)

    Set swDrawing = swModelDrawing

    swApp.ActivateDoc3 swModelDrawing.GetTitle, False, swRebuildOnActivation_e.swDontRebuildActiveDoc, lErr

    Set selMgr = swModelDrawing.SelectionManager
    swModelDrawing.ClearSelection2 True

    Debug.Assert False ' Select something in the drawing and press F5

    Set swView = swDrawing.FeatureByName("Drawing View1").GetSpecificFeature()
    Select Case selMgr.GetSelectedObjectType3(2, -1)
    Case swSelFACES, swSelEDGES, swSelVERTICES

        Set inputEntity = selMgr.GetSelectedObject6(2, -1)
        Set drComp = selMgr.GetSelectedObjectsComponent4(2, -1)

        Debug.Print "Using IModelDocExtension::GetCorrespondingEntity2()"
        Set outputEntity = swModelPart.Extension.GetCorrespondingEntity2(inputEntity)

        If outputEntity Is Nothing Then
            Debug.Print "No corresponding entity found in the part"
        Else
            Debug.Print "Corresponding entity found...selecting in part"
            swApp.ActivateDoc3 swModelPart.GetTitle, False, swRebuildOnActivation_e.swDontRebuildActiveDoc, lErr
            bSelected = outputEntity.Select4(False, Nothing)
        End If

    Case swSelNOTHING

    Case Else
        Set inputObject = selMgr.GetSelectedObject6(2, -1)
        Set drComp = selMgr.GetSelectedObjectsComponent4(2, -1)

        Debug.Print "Using IModelDocExtension::GetCorresponding2()"
        Set outputObject = swModelPart.Extension.GetCorresponding2(inputObject)

        If outputObject Is Nothing Then
            Debug.Print "No corresponding object found in the part"
        Else
            Debug.Print "Corresponding object found in the part"
        End If
    End Select
End Sub

```

## Get Corresponding Entities Between Parts and Drawing Views Example (VB.NET)

This example shows how to get corresponding entities or objects between a part and its drawing.

```

'-----
' Preconditions:
' 1. Ensure that the specified part and drawing exist.
' 2. Open the Immediate window.
' 3. Uncomment the subroutine you want to run in Main().
' 4. At the pause, select a face, edge, vertex, feature, annotation,
'    or sketch segment.
' 5. Press F5.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. If a corresponding face, edge, or vertex is found, it is selected in the
'    underlying part or drawing.
'
' NOTE: Because the models are used elsewhere, do not save changes.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Partial Class SolidWorksMacro
```

```

    Dim docSpec As DocumentSpecification
    Dim swModelPart As ModelDoc2
    Dim swModelDrawing As ModelDoc2
    Dim swDrawing As DrawingDoc
    Dim swView As View
    Dim lErr As Integer
    Dim selMgr As SelectionMgr
    Dim inputEntity As Entity
    Dim outputEntity As Entity
    Dim bSelected As Boolean
    Dim inputObject As Object
    Dim outputObject As Object
    Dim drComp As DrawingComponent

```

```
Public Sub Main()
```

```

    'Uncomment the subroutine you want to run; comment the other one
    'ViewToPart()
    PartToView()

```

```
End Sub
```

```
Public Sub ViewToPart()
```

```

    docSpec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.SLDPR")
    swModelPart = swApp.OpenDoc7(docSpec)

```

```

    docSpec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.SLDDRW")
    swModelDrawing = swApp.OpenDoc7(docSpec)

```

```

    swDrawing = swModelDrawing
    swView = swDrawing.FeatureByName("Drawing View1").GetSpecificFeature()

```

```
swApp.ActivateDoc3(swModelPart.GetTitle, True, swRebuildOnActivation_e.swUserDecision, lErr)
```

```

    selMgr = swModelPart.SelectionManager
    swModelPart.ClearSelection2(True)

```

```
Stop ' Select something in the model and press F5
```

```
Select Case selMgr.GetSelectedObjectType3(1, -1)
```

```
Case swSelectType_e.swSelFACES, swSelectType_e.swSelEDGES, swSelectType_e.swSelVERTICES
```

```
    inputEntity = selMgr.GetSelectedObject6(1, -1)
```

```
    Debug.Print("Using IView::GetCorrespondingEntity()")
```

```
    outputEntity = swView.GetCorrespondingEntity(inputEntity)
```

```

    If outputEntity Is Nothing Then
        Debug.Print("No corresponding entity found in the drawing view")
    Else

```

```

        Debug.Print("Corresponding entity found....selecting in drawing")
        swApp.ActivateDoc3(swModelDrawing.GetTitle, False, swRebuildOnActivation_e.swDontRebuildActiveDoc, lErr)
        bSelected = outputEntity.Select4(False, Nothing)
    End If

```

```
Case swSelectType_e.swSelNOTHING
```

```
Case Else
```

```
    inputObject = selMgr.GetSelectedObject6(1, -1)
```

```

        Debug.Print("Using IView::GetCorresponding()")
        outputObject = swView.GetCorresponding(inputObject)

        If outputObject Is Nothing Then
            Debug.Print("No corresponding object found in the drawing view")
        Else
            Debug.Print("Corresponding object found in the drawing view")
        End If
    End Select

End Sub

Public Sub PartToView()

    docSpec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.SLDPRPT")
    swModelPart = swApp.OpenDoc7(docSpec)

    docSpec = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\clamp1.SLDDRW")
    swModelDrawing = swApp.OpenDoc7(docSpec)

    swDrawing = swModelDrawing

    swApp.ActivateDoc3(swModelDrawing.GetTitle, False, swRebuildOnActivation_e.swDontRebuildActiveDoc, lErr)

    selMgr = swModelDrawing.SelectionManager
    swModelDrawing.ClearSelection2(True)

    Stop ' Select something in the drawing and press F5

    swView = swDrawing.FeatureByName("Drawing View1").GetSpecificFeature()

    Select Case selMgr.GetSelectedObjectType3(2, -1)
        Case swSelectType_e.swSelFACES, swSelectType_e.swSelEDGES, swSelectType_e.swSelVERTICES

            inputEntity = selMgr.GetSelectedObject6(2, -1)
            drComp = selMgr.GetSelectedObjectsComponent4(2, -1)

            Debug.Print("Using IModelDocExtension::GetCorrespondingEntity2()")

            outputEntity = swModelPart.Extension.GetCorrespondingEntity2(inputEntity)

            If outputEntity Is Nothing Then
                Debug.Print("No corresponding entity found in the part")
            Else
                Debug.Print("Corresponding entity found...selecting in part")
                swApp.ActivateDoc3(swModelPart.GetTitle, False, swRebuildOnActivation_e.swDontRebuildActiveDoc, lErr)
                bSelected = outputEntity.Select4(False, Nothing)
            End If

        Case swSelectType_e.swSelNOTHING

        Case Else

            inputObject = selMgr.GetSelectedObject6(2, -1)
            drComp = selMgr.GetSelectedObjectsComponent4(2, -1)

            Debug.Print("Using IModelDocExtension::GetCorresponding2()")

            outputObject = swModelPart.Extension.GetCorresponding2(inputObject)

            If outputObject Is Nothing Then
                Debug.Print("No corresponding object found in the part")
            Else
                Debug.Print("Corresponding object found in the part")
            End If
        End Select

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

## Get Corresponding Note in Assembly Example (VBA)

This example shows how to determine if the selected note in an assembly corresponds to the selected note in a part that is a component of the assembly.

```

'-----
' Preconditions:
' 1. Copy public_documents\samples\tutorial\api\block.sldprt to c:\temp.
' 2. Open c:\temp\block.sldprt and add a note containing the text Note1.
' 3. Save the part.
' 4. Create an assembly of the part.
' 5. Save the assembly as c:\temp\block.sldasm.
'
' Postconditions:
' 1. At Stop, select the note and press F5.
' 2. Gets the Note object in the assembly.
' 3. Activates the part document.
' 4. At Stop, select the note and press F5.
' 5. Gets the Note object in the assembly.
' 6. Re-activates the assembly.
' 7. Gets whether the notes in the part and assembly are the same.
' 8. Click OK to close the message box.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swAssembly As SldWorks.AssemblyDoc
Dim swPart As SldWorks.PartDoc
Dim swModel As SldWorks.ModelDoc2
Dim swCompModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swCompModelDocExt As SldWorks.ModelDocExtension
Dim swAnnotation As SldWorks.Annotation
Dim swAssemComp As SldWorks.Component2
Dim swAssemNoteCorresp As SldWorks.Note
Dim swAssyNote As SldWorks.Note
Dim swPartNote As SldWorks.Note
Dim boolstatus As Boolean
Dim nRetVal As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssembly = swModel
    Set swSelMgr = swModel.SelectionManager

    Stop
    ' Select the note in the assembly
    ' Press F5

    ' Get the note
    Set swAssyNote = swSelMgr.GetSelectedObject6(1, 0)
    swModel.ClearSelection2 True
    Set swAnnotation = swAssyNote.GetAnnotation
    boolstatus = swAnnotation.Select3(False, Nothing)

    ' Activate the part document
    Set swPart = swApp.ActivateDoc2("Block.SLDPRT", True, nRetVal)
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    swModel.ClearSelection2 True

    Stop
    ' Select the note
    ' Press F5

    ' Get the note
    Set swPartNote = swSelMgr.GetSelectedObject6(1, 0)
    swModel.ClearSelection2 True
    Set swAnnotation = swPartNote.GetAnnotation
    boolstatus = swAnnotation.Select3(False, Nothing)

    ' Re-activate the assembly
    Set swAssembly = swApp.ActivateDoc2("Block.SLDASM", True, nRetVal)
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    swModel.ClearSelection2 True

    ' Select the Block component in the assembly
    boolstatus = swModelDocExt.SelectByID2("Block-1@Block", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swAssemComp = swSelMgr.GetSelectedObject6(1, 0)
    Set swCompModel = swAssemComp.GetModelDoc
    Set swCompModelDocExt = swCompModel.Extension

    swModel.ClearSelection2 True

    ' Get whether the selected notes are the same note
    Set swAssemNoteCorresp = swCompModelDocExt.GetCorresponding(swPartNote)
    Dim strOutput As String

    ' Test feature returned by swCompModelDocExt::GetCorresponding
    If swAssemNoteCorresp Is Nothing Then
        strOutput = "Note not obtained."
    
```

```

Else
    If TypeOf swAssemNoteCorresp Is SldWorks.Note Then
        Set swAnnotation = swAssemNoteCorresp.GetAnnotation
        boolstatus = swAnnotation.Select3(False, Nothing)
        swAssemNoteCorresp.GetText

        'If swAssemNoteCorresp Is swAssyNote then
        'the pointers are different, but they point to the same note
        If swAssemNoteCorresp.GetText = "Note1" Then
            strOutput = "Note obtained correctly."
        Else
            strOutput = "swAssemNoteCorresp Is not swAssyNote."
        End If
    End If
End If

strOutput = strOutput + vbNewLine
swModel.ClearSelection2 True
strOutput = strOutput + vbNewLine
MsgBox strOutput

End Sub

```

## Get Corresponding Note in Part Example (VBA)

This example shows how to determine if the selected note in a part corresponds to the selected note in an assembly to which the part belongs.

```

'-----
' Preconditions:
' 1. Copy public_documents\samples\tutorial\api\block.sldprt to c:\temp.
' 2. Open c:\temp\block.sldprt and add a note containing the text Note1.
' 3. Save the part.
' 4. Create an assembly of the part.
' 5. Save the assembly as c:\temp\block.sldasm.
' 6. Open the Immediate window.
'
' Postconditions:
' 1. At Stop, select the note and press F5.
' 2. Gets the Note object in the assembly.
' 3. Activates the part document.
' 4. At Stop, select the note and press F5.
' 5. Gets the Note object in the assembly.
' 6. Gets whether the notes in the part and assembly are the same.
' 7. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swPart As SldWorks.PartDoc
Dim swAssemNote As SldWorks.Note
Dim swPartNote As SldWorks.Note
Dim swPartNoteCorresp As Object
Dim swAnnotation As SldWorks.Annotation
Dim status As Boolean
Dim errors As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    Stop
    ' Select Note1 and press F5

    Set swAssemNote = swSelMgr.GetSelectedObject6(1, 0)
    swModel.ClearSelection2 True

    ' Activate the part document
    Set swPart = swApp.ActivateDoc3("Block.SLDPRT", True, swRebuildOnActivation_e.swUserDecision, errors)
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swPart.Extension

    swModel.ClearSelection2 True

    Stop
    ' Select Note1 and press F5

    Set swPartNote = swSelMgr.GetSelectedObject6(1, 0)

    ' Get whether the selected notes are the same note
    Set swPartNoteCorresp = swModelDocExt.GetCorresponding(swAssemNote)
    If swPartNoteCorresp Is Nothing Then
        Debug.Print ("Note not obtained.")
    Else
        If TypeOf swPartNoteCorresp Is SldWorks.Note Then
            Set swAnnotation = swPartNoteCorresp.GetAnnotation
            swModel.ClearSelection2 True
            status = swAnnotation.Select3(False, Nothing)
            swPartNoteCorresp.GetText
            ' If swAssemNoteCorresp is swAssyNote, then
            ' the pointers are different, but they point to the same note
            If swPartNoteCorresp.GetText = "Note1" Then
                Debug.Print ("Note obtained correctly.")
            Else
                Debug.Print ("swPartNoteCorresp Is Not swAssyNote.")
            End If
        End If
    End If
End Sub

```

## Get Corresponding Objects in Assembly Component Example (C#)

This example shows how to get the corresponding sketch contour, sketch segments, and annotation for a component in the context of the assembly.

```
//-----
// Preconditions:
// 1. Verify that:
//    * specified part and assembly templates
//    * C:\test
//    exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part and creates a sketch containing
//    a sketch arc, sketch line, and a note.
// 2. Saves the part as C:\test\part1.sldprt.
// 3. Makes an assembly using the part document and saves
//    the assembly as C:\test\assem1.sldasm.
// 4. Activates the part.
//    a. Gets the persistent reference IDs of the sketch segments
//       in the sketch contour.
//    b. Gets the object ID of the note annotation.
// 5. Activates the assembly.
//    a. Gets the persistent reference IDs of the sketch
//       segments in the sketch contour in the context
//       of the assembly.
//    b. Gets the object ID of the note annotation in the context
//       of the assembly.
// 6. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            Component2 swComponent = default(Component2);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            Note swNote = default(Note);
            Annotation swAnnotation = default(Annotation);
            TextFormat swTextFormat = default(TextFormat);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeature = default(Feature);
            Sketch swSketch = default(Sketch);
            SketchContour swSketchContour = null;
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            object swObject = null;
            object[] sketchSegments = null;
            object[] sketchContours = null;
            int nbrSketchContours = 0;
            int nbrSketchSegments = 0;
            int[] sketchSegmentIDs = null;
            int sketchSegmentType = 0;
            int annotationID = 0;
            int annotationType = 0;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            int i = 0;
            int j = 0;

            //Create sketch containing a sketch arc,
            //sketch line, and annotation
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2016\\templates\\Part.prtdot", 0, 0, 0);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchManager.CreateArc(-0.0756, 0.0, 0.0, -0.020568, 0.0, 0.0, -0.130614, 0.001423, 0.0, 1);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.130614, 0.001423, 0.0, -0.0756, -0.047042, 0.0);
            swNote = (Note)swModel.InsertNote("This is a sketch segment");
            if ((swNote != null))
            {
                swNote.LockPosition = false;
                swNote.Angle = 0;
                status = swNote.SetBalloon(0, 0);
                swAnnotation = (Annotation)swNote.GetAnnotation();
                if ((swAnnotation != null))
                {
                    errors = swAnnotation.SetLeader3((int)swLeaderStyle_e.swUNDERLINED, 0, true, false, false, false);
                    status = swAnnotation.SetPosition2(-0.002501468059071, 0.0826874163970699, 0);
                    swTextFormat = null;
                    status = swAnnotation.SetTextFormat(0, true, swTextFormat);
                }
            }
            swModel.ClearSelection2(true);
            swModel.WindowRedraw();
            swSketchManager.InsertSketch(true);
            swModelDocExt = swModel.Extension;
            status = swModelDocExt.SaveAs("C:\\test\\part1.sldprt", (int)swSaveAsVersion_e.swSaveAsCurrentVersion, (int)swSaveAsOptions_e.swSaveAsOptions_
            Silent, null, ref errors, ref warnings);

            //Save part as assembly
            swAssembly = (AssemblyDoc)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2016\\templates\\Assembly.asmdot", 0, 0, 0);
            swComponent = (Component2)swAssembly.AddComponent5("C:\\test\\part1.SLDPRT", (int)swAddComponentConfigOptions_e.swAddComponentConfigOptions_Cu
            rrentSelectedConfig, "", false, "", 1.60609059776107E-05, 0, 8.47512097834624E-06);
            swModel = (ModelDoc2)swAssembly;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SaveAs("C:\\test\\Assem1.SLDASM", (int)swSaveAsVersion_e.swSaveAsCurrentVersion, (int)swSaveAsOptions_e.swSaveAsOptions_
            _Silent, null, ref errors, ref warnings);

            //Get persistent reference IDs of sketch segments in sketch contour in part
            swModel = (ModelDoc2)swApp.ActivateDoc3("Part1.SLDPRT", false, (int)swRebuildOnActivation_e.swRebuildActiveDoc, ref errors);
```



```

swModelDocExt = (ModelDocExtension)swModel.Extension;
status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0.0, 0.0, 0.0, false, 0, null, 0);
swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
swSketch = (Sketch)swFeature.GetSpecificFeature2();
Debug.Print(swModel.GetPathName());
Debug.Print("");
if ((swSketch != null))
{
    sketchContours = (object[])swSketch.GetSketchContours();
    nbrSketchContours = sketchContours.Length;
    Debug.Print(" Number of sketch contours in " + swFeature.Name + " = " + nbrSketchContours);
    for (i = 0; i < sketchContours.Length; i++)
    {
        swSketchContour = (SketchContour)sketchContours[i];
        if ((swSketchContour != null))
        {
            status = swSketchContour.Select2(false, null);
        }
        sketchSegments = (object[])swSketchContour.GetSketchSegments();
        nbrSketchSegments = sketchSegments.Length;
        for (j = 0; j < sketchSegments.Length; j++)
        {
            swSketchSegment = (SketchSegment)sketchSegments[j];
            if ((swSketchSegment != null))
            {
                sketchSegmentIDs = (int[])swSketchSegment.GetID();
                sketchSegmentType = swSketchSegment.GetType();
                Debug.Print(" Persistent IDs = [" + sketchSegmentIDs[0] + ", " + sketchSegmentIDs[1] + "] and type = " + sketchSegmentType +
" (0 = line; 1 = arc)");
            }
        }
    }
}

//Get object ID of note annotation in part
status = swModelDocExt.SelectByID2("DetailItem1@Annotations", "NOTE", -0.00650517330771608, 0.0568327787544409, -0.035178659814812, false, 0,
null, 0);

swNote = (Note)swSelectionMgr.GetSelectedObject6(1, -1);
swAnnotation = (Annotation)swNote.GetAnnotation();
annotationType = swAnnotation.GetType();
annotationID = swModelDocExt.GetObjectID(swAnnotation);
Debug.Print("");
Debug.Print(" Annotation ID = " + annotationID + " and type = " + annotationType + " (6 = note)");

//Activate the assembly
swModel = (ModelDoc2)swApp.ActivateDoc3("Assem1.SLDASM", false, (int)swRebuildOnActivation_e.swRebuildActiveDoc, ref errors);
swModelDocExt = (ModelDocExtension)swModel.Extension;
status = swModelDocExt.SelectByID2("Part1-1@Assem1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
swComponent = (Component2)swSelectionMgr.GetSelectedObject6(1, -1);
Debug.Print("");
Debug.Print(swModel.GetPathName());
Debug.Print("");

//Get corresponding sketch contour and sketch segments
//and their persistent reference IDs in component
swObject = (object)swComponent.GetCorresponding(swSketchContour);
swSketchContour = null;
swSketchContour = (SketchContour)swObject;
if ((swSketchContour != null))
{
    status = swSketchContour.Select2(false, null);
}
sketchSegments = (object[])swSketchContour.GetSketchSegments();
Debug.Print(" Number of sketch contours in " + swFeature.Name + " = " + nbrSketchContours);
nbrSketchSegments = sketchSegments.Length;
for (j = 0; j < sketchSegments.Length; j++)
{
    swSketchSegment = (SketchSegment)sketchSegments[j];
    if ((swSketchSegment != null))
    {
        sketchSegmentIDs = (int[])swSketchSegment.GetID();
        sketchSegmentType = swSketchSegment.GetType();
        Debug.Print(" Persistent IDs = [" + sketchSegmentIDs[0] + ", " + sketchSegmentIDs[1] + "] and type = " + sketchSegmentType + " (0 = l
ine; 1 = arc)");
    }
}

//Get object ID of corresponding note annotation in component
swObject = (object)swComponent.GetCorresponding(swNote);
swNote = null;
swNote = (Note)swObject;
if ((swNote != null))
{
    swAnnotation = (Annotation)swNote.GetAnnotation();
    annotationType = swAnnotation.GetType();
    annotationID = swModelDocExt.GetObjectID(swAnnotation);
    Debug.Print("");
    Debug.Print(" Annotation ID = " + annotationID + " and type = " + annotationType + " (6 = note)");
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

## Get Corresponding Objects in Assembly Component Example (VBA)

This example shows how to get the corresponding sketch contour, sketch segments, and annotation for a component in the context of the assembly.

```

'-----
' Preconditions:
' 1. Verify that:
'   * specified part and assembly templates
'   * C:\test
'   exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part and creates a sketch containing
'   a sketch arc, sketch line, and a note.
' 2. Saves the part as C:\test\part1.sldprt.
' 3. Makes an assembly using the part document and saves
'   the assembly as C:\test\assem1.sldasm.
' 4. Activates the part.
'   a. Gets the persistent reference IDs of the sketch segments
'      in the sketch contour.
'   b. Gets the object ID of the note annotation.
' 5. Activates the assembly.
'   a. Gets the persistent reference IDs of the sketch
'      segments in the sketch contour in the context
'      of the assembly.
'   b. Gets the object ID of the note annotation in the context
'      of the assembly.
' 6. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssembly As SldWorks.AssemblyDoc
Dim swComponent As SldWorks.Component2
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swNote As SldWorks.Note
Dim swAnnotation As SldWorks.Annotation
Dim swTextFormat As SldWorks.TextFormat
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeature As SldWorks.Feature
Dim swSketch As SldWorks.Sketch
Dim swSketchContour As SldWorks.SketchContour
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swObject As Object
Dim sketchSegments As Variant
Dim sketchContours As Variant
Dim nbrSketchContours As Long
Dim nbrSketchSegments As Long
Dim sketchSegmentIDs As Variant
Dim sketchSegmentType As Long
Dim annotationID As Long
Dim annotationType As Long
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim i As Long
Dim j As Long

Sub main()

    Set swApp = Application.SldWorks

    'Create sketch containing a sketch arc,
    'sketch line, and annotation
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Part.prtdot", 0, 0, 0)
    Set swSketchManager = swModel.SketchManager
    Set swSketchSegment = swSketchManager.CreateArc(-0.0756, 0#, 0#, -0.020568, 0#, 0#, -0.130614, 0.001423, 0#, 1)
    Set swSketchSegment = swSketchManager.CreateLine(-0.130614, 0.001423, 0#, -0.0756, -0.047042, 0#)
    Set swNote = swModel.InsertNote("This is a sketch segment")
    If Not swNote Is Nothing Then
        swNote.LockPosition = False
        swNote.Angle = 0
        status = swNote.SetBalloon(0, 0)
        Set swAnnotation = swNote.GetAnnotation()
        If Not swAnnotation Is Nothing Then
            errors = swAnnotation.SetLeader3(swLeaderStyle_e.swUNDERLINED, 0, True, False, False, False)
            status = swAnnotation.SetPosition2(-0.002501468059071, 8.26874163970699E-02, 0)
            status = swAnnotation.SetTextFormat(0, True, swTextFormat)
        End If
    End If
    Set swModel.ClearSelection2 True
    swModel.WindowRedraw
    swSketchManager.InsertSketch True
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SaveAs("C:\test\Part1.SLDPR1", swSaveAsVersion_e.swSaveAsCurrentVersion, swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing,
errors, warnings)

    'Save part as assembly
    Set swAssembly = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Assembly.asmdot", 0, 0, 0)
    Set swComponent = swAssembly.AddComponent5("C:\test\Part1.SLDPR1", swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig,
"", False, "", -1.60609059776107E-05, 0, 8.47512097834624E-06)
    Set swModel = swAssembly
    Set swModelDocExt = swModel.Extension

```

```

status = swModelDocExt.SaveAs("C:\test\Assem1.SLDASM", swSaveAsVersion_e.swSaveAsCurrentVersion, swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing,
errors, warnings)

'Get persistent reference IDs of sketch segments in sketch contour in part
Set swModel = swApp.ActivateDoc3("Part1.SLDPRT", False, swRebuildOnActivation_e.swRebuildActiveDoc, errors)
Set swModelDocExt = swModel.Extension
status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0#, 0#, 0#, False, 0, Nothing, 0)
Set swSelectionMgr = swModel.SelectionManager
Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
Set swSketch = swFeature.GetSpecificFeature2()
Debug.Print swModel.GetPathName
Debug.Print ""
If Not swSketch Is Nothing Then
    sketchContours = swSketch.GetSketchContours()
    nbrSketchContours = UBound(sketchContours) - LBound(sketchContours) + 1
    Debug.Print " Number of sketch contours in " & swFeature.Name & " = " & nbrSketchContours
    For i = LBound(sketchContours) To UBound(sketchContours)
        Set swSketchContour = sketchContours(i)
        If Not swSketchContour Is Nothing Then
            status = swSketchContour.Select2(False, Nothing)
        End If
        sketchSegments = swSketchContour.GetSketchSegments()
        nbrSketchSegments = UBound(sketchSegments) - LBound(sketchSegments) + 1
        For j = LBound(sketchSegments) To UBound(sketchSegments)
            Set swSketchSegment = sketchSegments(j)
            If Not swSketchSegment Is Nothing Then
                sketchSegmentIDs = swSketchSegment.GetID
                sketchSegmentType = swSketchSegment.GetType
                Debug.Print " Persistent IDs = [" & sketchSegmentIDs(0) & ", " & sketchSegmentIDs(1) & "] and type = " & sketchSegmentType & " (0
= line; 1 = arc)"
            End If
        Next
    Next
End If

'Get object ID of note annotation in part
status = swModelDocExt.SelectByID2("DetailItem1@Annotations", "NOTE", -6.50517330771608E-03, 5.68327787544409E-02, -0.035178659814812, False, 0,
Nothing, 0)
Set swNote = swSelectionMgr.GetSelectedObject6(1, -1)
Set swAnnotation = swNote.GetAnnotation
annotationType = swAnnotation.GetType
annotationID = swModelDocExt.GetObjectID(swAnnotation)
Debug.Print ""
Debug.Print " Annotation ID = " & annotationID & " and type = " & annotationType & " (6 = note)"

'Activate the assembly
Set swModel = swApp.ActivateDoc3("Assem1.SLDASM", False, swRebuildOnActivation_e.swRebuildActiveDoc, errors)
Set swModelDocExt = swModel.Extension
status = swModelDocExt.SelectByID2("Part1-1@Assem1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
Set swSelectionMgr = swModel.SelectionManager
Set swComponent = swSelectionMgr.GetSelectedObject6(1, -1)
Debug.Print ""
Debug.Print swModel.GetPathName
Debug.Print ""
'Get corresponding sketch contour and sketch segments
'and their persistent reference IDs in component
Set swObject = swComponent.GetCorresponding(swSketchContour)
Set swSketchContour = Nothing
Set swSketchContour = swObject
If Not swSketchContour Is Nothing Then
    status = swSketchContour.Select2(False, Nothing)
End If
sketchSegments = swSketchContour.GetSketchSegments()
Debug.Print " Number of sketch contours in " & swFeature.Name & " = " & nbrSketchContours
nbrSketchSegments = UBound(sketchSegments) - LBound(sketchSegments) + 1
For j = 0 To UBound(sketchSegments)
    Set swSketchSegment = sketchSegments(j)
    If Not swSketchSegment Is Nothing Then
        sketchSegmentIDs = swSketchSegment.GetID
        sketchSegmentType = swSketchSegment.GetType
        Debug.Print " Persistent IDs = [" & sketchSegmentIDs(0) & ", " & sketchSegmentIDs(1) & "] and type = " & sketchSegmentType & " (0 = line; 1 =
arc)"
    End If
Next

'Get object ID of corresponding note annotation in component
Set swObject = swComponent.GetCorresponding(swNote)
Set swNote = Nothing
Set swNote = swObject
If Not swNote Is Nothing Then
    Set swAnnotation = swNote.GetAnnotation
    annotationType = swAnnotation.GetType
    annotationID = swModelDocExt.GetObjectID(swAnnotation)
    Debug.Print ""
    Debug.Print " Annotation ID = " & annotationID & " and type = " & annotationType & " (6 = note)"
End If

End Sub

```

## Get Corresponding Objects in Assembly Component Example (VB.NET)

This example shows how to get the corresponding sketch contour, sketch segments, and annotation for a component in the context of the assembly.

```

-----
Preconditions:
1. Verify that:
    * specified part and assembly templates
    * C:\test
    exist.
2. Open the Immediate window.

Postconditions:
1. Opens a new part and creates a sketch containing
   a sketch arc, sketch line, and a note.
2. Saves the part as C:\test\part1.sldprt.
3. Makes an assembly using the part document and saves
   the assembly as C:\test\assem1.sldasm.
4. Activates the part.
   a. Gets the persistent reference IDs of the sketch segments
      in the sketch contour.
   b. Gets the object ID of the note annotation.
5. Activates the assembly.
   a. Gets the persistent reference IDs of the sketch
      segments in the sketch contour in the context
      of the assembly.
   b. Gets the object ID of the note annotation in the context
      of the assembly.
6. Examine the Immediate window.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swAssembly As AssemblyDoc
        Dim swComponent As Component2
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swNote As Note
        Dim swAnnotation As Annotation
        Dim swTextFormat As TextFormat
        Dim swModelDocExt As ModelDocExtension
        Dim swFeature As Feature
        Dim swSketch As Sketch
        Dim swSketchContour As SketchContour = Nothing
        Dim swSelectionMgr As SelectionMgr
        Dim swObject As Object
        Dim sketchSegments() As Object
        Dim sketchContours() As Object
        Dim nbrSketchContours As Integer
        Dim nbrSketchSegments As Integer
        Dim sketchSegmentIDs() As Integer
        Dim sketchSegmentType As Integer
        Dim annotationID As Integer
        Dim annotationType As Integer
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim i As Integer
        Dim j As Integer

        'Create sketch containing a sketch arc,
        'sketch line, and annotation
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Part.prtdot", 0, 0, 0)
        swSketchManager = swModel.SketchManager
        swSketchSegment = swSketchManager.CreateArc(-0.0756, 0.0#, 0.0#, -0.020568, 0.0#, 0.0#, -0.130614, 0.001423, 0.0#, 1)
        swSketchSegment = swSketchManager.CreateLine(-0.130614, 0.001423, 0.0#, -0.0756, -0.047042, 0.0#)
        swNote = swModel.InsertNote("This is a sketch segment")
        If Not swNote Is Nothing Then
            swNote.LockPosition = False
            swNote.Angle = 0
            status = swNote.SetBalloon(0, 0)
            swAnnotation = swNote.GetAnnotation()
            If Not swAnnotation Is Nothing Then
                errors = swAnnotation.SetLeader3(swLeaderStyle_e.swUNDERLINED, 0, True, False, False, False)
                status = swAnnotation.SetPosition2(-0.002501468059071, 0.0826874163970699, 0)
                swTextFormat = Nothing
                status = swAnnotation.SetTextFormat(0, True, swTextFormat)
            End If
        End If
        swModel.ClearSelection2(True)
        swModel.WindowRedraw()
        swSketchManager.InsertSketch(True)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SaveAs("C:\test\Part1.SLDPRT", swSaveAsVersion_e.swSaveAsCurrentVersion, swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing, errors, warnings)

        'Save part as assembly
        swAssembly = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Assembly.asmdot", 0, 0, 0)
        swComponent = swAssembly.AddComponent5("C:\test\Part1.SLDPRT", swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "", 0.0000160609059776107, 0, 0.00000847512097834624)
        swModel = swAssembly
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SaveAs("C:\test\Assem1.SLDASM", swSaveAsVersion_e.swSaveAsCurrentVersion, swSaveAsOptions_e.swSaveAsOptions_Silent, Nothing, errors, warnings)

        'Get persistent reference IDs of sketch segments in sketch contour in part
        swModel = swApp.ActivateDoc3("Part1.SLDPRT", False, swRebuildOnActivation_e.swRebuildActiveDoc, errors)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0.0#, 0.0#, 0.0#, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swSketch = swFeature.GetSpecificFeature2()
    
```

```

Debug.Print(swModel.GetPathName)
Debug.Print("")
If Not swSketch Is Nothing Then
    sketchContours = swSketch.GetSketchContours()
    nbrSketchContours = UBound(sketchContours) - LBound(sketchContours) + 1
    Debug.Print(" Number of sketch contours in " & swFeature.Name & " = " & nbrSketchContours)
    For i = LBound(sketchContours) To UBound(sketchContours)
        swSketchContour = sketchContours(i)
        If Not swSketchContour Is Nothing Then
            status = swSketchContour.Select2(False, Nothing)
        End If
        sketchSegments = swSketchContour.GetSketchSegments()
        nbrSketchSegments = UBound(sketchSegments) - LBound(sketchSegments) + 1
        For j = LBound(sketchSegments) To UBound(sketchSegments)
            swSketchSegment = sketchSegments(j)
            If Not swSketchSegment Is Nothing Then
                sketchSegmentIDs = swSketchSegment.GetID
                sketchSegmentType = swSketchSegment.GetType
                Debug.Print(" Persistent IDs = [" & sketchSegmentIDs(0) & ", " & sketchSegmentIDs(1) & "]" and type = " & sketchSegmentType & " (0
= line; 1 = arc)")
            End If
        Next
    Next
End If

'Get object ID of note annotation in part
status = swModelDocExt.SelectByID2("DetailItem1@Annotations", "NOTE", -0.00650517330771608, 0.0568327787544409, -0.035178659814812, False, 0, Noth
ing, 0)
swNote = swSelectionMgr.GetSelectedObject6(1, -1)
swAnnotation = swNote.GetAnnotation
annotationType = swAnnotation.GetType
annotationID = swModelDocExt.GetObjectID(swAnnotation)
Debug.Print("")
Debug.Print(" Annotation ID = " & annotationID & " and type = " & annotationType & " (6 = note)")

'Activate the assembly
swModel = swApp.ActivateDoc3("Assem1.SLDASM", False, swRebuildOnActivation_e.swRebuildActiveDoc, errors)
swModelDocExt = swModel.Extension
status = swModelDocExt.SelectByID2("Part1-1@Assem1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
swSelectionMgr = swModel.SelectionManager
swComponent = swSelectionMgr.GetSelectedObject6(1, -1)
Debug.Print("")
Debug.Print(swModel.GetPathName)
Debug.Print("")

'Get corresponding sketch contour and sketch segments
'and their persistent reference IDs in component
swObject = swComponent.GetCorresponding(swSketchContour)
swSketchContour = Nothing
swSketchContour = swObject
If Not swSketchContour Is Nothing Then
    status = swSketchContour.Select2(False, Nothing)
End If
sketchSegments = swSketchContour.GetSketchSegments()
Debug.Print(" Number of sketch contours in " & swFeature.Name & " = " & nbrSketchContours)
nbrSketchSegments = UBound(sketchSegments) - LBound(sketchSegments) + 1
For j = 0 To UBound(sketchSegments)
    swSketchSegment = sketchSegments(j)
    If Not swSketchSegment Is Nothing Then
        sketchSegmentIDs = swSketchSegment.GetID
        sketchSegmentType = swSketchSegment.GetType
        Debug.Print(" Persistent IDs = [" & sketchSegmentIDs(0) & ", " & sketchSegmentIDs(1) & "]" and type = " & sketchSegmentType & " (0 = line;
1 = arc)")
    End If
Next

'Get object ID of corresponding note annotation in component
swObject = swComponent.GetCorresponding(swNote)
swNote = Nothing
swNote = swObject
If Not swNote Is Nothing Then
    swAnnotation = swNote.GetAnnotation
    annotationType = swAnnotation.GetType
    annotationID = swModelDocExt.GetObjectID(swAnnotation)
    Debug.Print("")
    Debug.Print(" Annotation ID = " & annotationID & " and type = " & annotationType & " (6 = note)")
End If

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

## Get Cosmetic Threads Features in a Part Example (VBA)

This example shows how to create and get mirrored and patterned cosmetic thread feature data in a part document.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\holecube.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Mirrors and patterns the cosmetic thread feature.
' 2. Examine the Immediate window.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swMathUtility As SldWorks.MathUtility
Dim boolstatus As Boolean

Sub main()

    Dim myModel As SldWorks.ModelDoc2
    Dim thisFeature As SldWorks.Feature
    Dim thisSubFeature As SldWorks.Feature

    Set swApp = Application.SldWorks
    Set swMathUtility = swApp.GetMathUtility()
    Set myModel = swApp.ActiveDoc

    'Mirror and pattern the cosmetic thread subfeature
    boolstatus = myModel.Extension.SelectByID("", "EDGE", -8.02489357837999E-04, -2.46888176810671E-02, 6.00726028778809E-05, True, 0, Nothing, 0)
    Set thisFeature = myModel.FeatureManager.InsertCosmeticThread3(swStandardType_StandardHelicoilMetric, "Helicoil threads", "M33x2.0", 0.033,
swEndConditionBlind2Dia, 0.025, "M33x2.0 Helicoil Threads")
    boolstatus = myModel.Extension.SelectByID("Cut-Extrude1", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)
    boolstatus = myModel.Extension.SelectByID("", "FACE", -5.25693542039818E-02, -7.07674502874625E-02, 0, True, 128, Nothing, 0)
    boolstatus = myModel.Extension.SelectByID("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, True, 4, Nothing, 0)
    boolstatus = myModel.Extension.SelectByID("", "EDGE", -7.36957308265147E-02, -4.66691871832836E-02, 1.25295787199775E-04, True, 1, Nothing, 0)
    myModel.FeatureManager.FeatureCircularPattern4 3, 1.25663706143592, True, "NULL", False, False, False

    ' Traverse features and subfeatures of this model and look for cosmetic threads
    Set thisFeature = myModel.FirstFeature
    While Not thisFeature Is Nothing
        Set thisSubFeature = thisFeature.GetFirstSubFeature()
        While Not thisSubFeature Is Nothing
            If (thisSubFeature.GetTypeName() = "CosmeticThread") Then
                Debug.Print "Processing " & thisSubFeature.Name
                Call processCosmeticThread(myModel, thisSubFeature)
            End If
            Set thisSubFeature = thisSubFeature.GetNextSubFeature()
        Wend
        Set thisFeature = thisFeature.GetNextFeature()
    Wend
End Sub

Private Sub processCosmeticThread(myModel As SldWorks.ModelDoc2, aFeature As SldWorks.Feature)

    Dim thisCThread As SldWorks.CosmeticThreadFeatureData
    Dim patternedCount As Long
    Dim holeEdge As SldWorks.Edge
    Dim holeCurve As SldWorks.Curve
    Dim vCurveParams As Variant
    Dim basePt(0 To 2) As Double
    Dim vBasePt As Variant
    Dim mBasePt As SldWorks.MathPoint
    Dim edgeEntity As SldWorks.Entity
    Dim selData As SldWorks.SelectData
    Dim vPatternedXform As Variant
    Dim i As Integer
    Dim transform As SldWorks.MathTransform
    Dim mTransPt As SldWorks.MathPoint
    Dim vTransPt As Variant
    Dim transPtX As Double, transPtY As Double, transPtZ As Double
    Dim append As Boolean, myCallout As SldWorks.Callout

    Set thisCThread = aFeature.GetDefinition()

    If Not thisCThread Is Nothing Then
        ' Retrieve the information about the edge associated with the cosmetic thread
        boolstatus = thisCThread.AccessSelections(myModel, Nothing)
        Set holeEdge = thisCThread.Edge()
        If Not holeEdge Is Nothing Then
            Set holeCurve = holeEdge.GetCurve()
            If Not holeCurve Is Nothing Then
                vCurveParams = holeEdge.GetCurveParams2()
                basePt(0) = vCurveParams(0)
                basePt(1) = vCurveParams(1)
                basePt(2) = vCurveParams(2)
                Debug.Print "    0 (" & Format(basePt(0), "###0.0#####") & ", " & Format(basePt(1), "###0.0#####") & ", " & Format(basePt(2),
"###0.0#####") & ")"
            End If
        End If
        thisCThread.ReleaseSelectionAccess
        vBasePt = basePt
    End If

```

```

Set mBasePt = swMathUtility.CreatePoint((vBasePt))

' Select the edge used for the cosmetic thread
Set edgeEntity = holeEdge
append = True
boolstatus = edgeEntity.Select4(append, selData)

' Retrieve information about any patterns made from this cosmetic thread
patternedCount = thisCThread.GetPatternedTransformsCount()
Debug.Print "          Pattern count = " & patternedCount
vPatternedXform = thisCThread.PatternedTransforms()
If Not IsEmpty(vPatternedXform) Then
    For i = LBound(vPatternedXform) To UBound(vPatternedXform)
        Set transform = vPatternedXform(i)
        Set mTransPt = mBasePt.MultiplyTransform(transform)
        vTransPt = mTransPt.ArrayData()
        transPtX = vTransPt(0)
        transPtY = vTransPt(1)
        transPtZ = vTransPt(2)
        Debug.Print "          Pattern " & Str(i + 1) & " (" & Format(transPtX, "###0.0#####") & ", " & Format(transPtY, "###0.0#####") & ", " & Format(transPtZ, "###0.0#####") & ")"
        ' The transform information should be sufficient
        ' for getting the necessary geometry
        ' information for this cosmetic thread and
        ' its patterned cosmetic threads
        ' The next step attempts to select the edge used for
        ' patterned cosmetic threads to help verify that the
        ' transform information is accurate and that
        ' an edge could be obtained, if necessary
        ' Selections depend on whether the edge is
        ' actually visible in this orientation and this display state
        append = True
        boolstatus = myModel.Extension.SelectByID("", "EDGE", transPtX, transPtY, transPtZ, append, 0, myCallout, 0)
        If (boolstatus = 0) Then
            Debug.Print "          Selection failed?"
        End If
    Next i
End If
End If
End Sub

```

## Get COSMOSMotion Motion Study Properties and Results Example (VBA)

This example shows how to get a COSMOSMotion motion study's properties and results.

```
'-----
' Preconditions:
' 1. Open a model that has a COSMOSMotion motion study
'    named PhysSim and at least one COSMOSMotion motion
'    study plot.
' 2. In SOLIDWORKS Design, load the SOLIDWORKS Motion Study add-in
'    (click Tools > Add-Ins > SOLIDWORKS Motion).
' 3. In the SOLIDWORKS VBA IDE, add a reference to the
'    SolidWorks MotionStudy type library.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Gets the MotionManager.
' 2. Gets and activates the motion study named PhysSim.
' 3. Gets some motion study properties, number of plots,
'    motion study results, and whether the motion study results
'    are out of date.
' 4. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMotionMgr As SwMotionStudy.MotionStudyManager
Dim swMotionStudy1 As SwMotionStudy.MotionStudy
Dim swResults As SwMotionStudy.MotionStudyResults
Dim swMotionStudyProps As SwMotionStudy.MotionStudyProperties
Dim swCosmosMotionStudyProps As SwMotionStudy.CosmosMotionStudyProperties
Dim swCosmosMotionStudyResults As SwMotionStudy.CosmosMotionStudyResults

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Get the MotionManager
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()
    If (swMotionMgr Is Nothing) Then
        End
    End If

    ' Get motion study named PhysSim
    Set swMotionStudy1 = swMotionMgr.GetMotionStudy("PhysSim")
    If (swMotionStudy1 Is Nothing) Then
        MsgBox "PhysSim is not available."
        End
    End If

    swMotionStudy1.StudyType = swMotionStudyType_e.swMotionStudyTypePhysicalSimulation

    'Activate motion study
    If Not swMotionStudy1.IsActive Then swMotionStudy1.Activate
    Debug.Print "Duration of motion study: " & swMotionStudy1.GetDuration
    Debug.Print "Timebar position on timeline: " & swMotionStudy1.GetTime
    Debug.Print "Number of motion features: " & swMotionStudy1.GetMotionFeaturesCount

    'Get COSMOSMotion study properties
    Set swMotionStudyProps = swMotionStudy1.GetProperties(4)
    Set swCosmosMotionStudyProps = swMotionStudyProps

    Debug.Print ""
    Debug.Print "COSMOSMotion study properties: "
    Debug.Print "    Animate during simulation: " & swCosmosMotionStudyProps.AnimateDuringSimulation
    Debug.Print "    Make all mates flexible: " & swCosmosMotionStudyProps.MakeAllMatesFlexible
    Debug.Print "    Display results graphics as wireframe: " & swCosmosMotionStudyProps.WireframeGraphics

    ' Get COSMOSMotion study results
    Set swResults = swMotionStudy1.GetResults(4)
    Set swCosmosMotionStudyResults = swResults
    Debug.Print "    Number of plots: " & swCosmosMotionStudyResults.GetPlotCount
    Debug.Print ""
    Debug.Print "Are the motion study results out of date? " & swResults.IsOutOfDate

End Sub
```



## Get Cross Break Feature Data in Sheet Metal Part Example (C#)

This example shows how to get cross break feature data in a sheet metal part.

```
//-----
// Preconditions:
// 1. Verify that the specified part document template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a sheet metal part with a cross break feature.
// 2. Iterates over all features in the part and gets the
//    cross break feature.
// 3. Names the cross break feature's face.
// 4. Prints to the Immediate window this cross break
//    feature data:
//    * Name of the face
//    * Whether its direction is reversed
//    * Radius
//    * Angle
// 5. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            PartDoc swPart = default(PartDoc);
            ModelDoc2 swModel = default(ModelDoc2);
            SketchManager swSketchManager = default(SketchManager);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            CustomBendAllowance swCustomBendAllowance = default(CustomBendAllowance);
            Feature swFeature = default(Feature);
            CrossBreakFeatureData swCrossBreakFeatureData = default(CrossBreakFeatureData);
            Face2 swFace = default(Face2);
            Entity swEntity = default(Entity);
            object[] sketchLines = null;
            object[] features = null;
            string faceName = null;
            bool status = false;
            int i = 0;

            //Create sheet metal part with cross break feature
            swPart = (PartDoc)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\part.prtdot", 0, 0, 0);
            swModel = (ModelDoc2)swPart;
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, true);
            sketchLines = (object[])swSketchManager.CreateCornerRectangle(0, 0, 0, 0.112582864792503, -0.0690084337349398, 0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);
            swModel.ShowNamedView2("Trimetric", 8);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swCustomBendAllowance = (CustomBendAllowance)swFeatureManager.CreateCustomBendAllowance();
            swCustomBendAllowance.KFactor = 0.5;
            swFeature = (Feature)swFeatureManager.InsertSheetMetalBaseFlange2(0.0007366, false, 0.01905, 0.00508, 0.00254, false, 0, 0, 1, swCustomBendAllowance, false, 0, 0.0001, 0.0001, 0.5, true, false, true, true);
            status = swModelDocExt.SelectByID2("Base-Flange1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("", "FACE", 0.0440948432654409, -0.0302769643316765, 0, true, 0, null, 0);
            swFeature = (Feature)swFeatureManager.InsertCrossBreak(1.5707963267949, 0.000635);
            swModel.ClearSelection2(true);

            //Get the cross break feature
            //by iterating over all features
            features = (object[])swFeatureManager.GetFeatures(true);
            for (i = 0; i < features.Length; i++)
            {
                swFeature = (Feature)features[i];
                if (swFeature.GetTypeName2() == "CrossBreak")
                {
                    swCrossBreakFeatureData = (CrossBreakFeatureData)swFeature.GetDefinition();
                    swCrossBreakFeatureData.AccessSelections(swModel, null);
                    swFace = (Face2)swCrossBreakFeatureData.Face;
                    swEntity = (Entity)swFace;
                    faceName = "CrossBreakFace";
                    status = swPart.SetEntityName(swEntity, faceName);
                    faceName = swModel.GetEntityName(swEntity);
                    Debug.Print("Cross break feature data:");
                    Debug.Print("  Name of face: " + faceName);
                    Debug.Print("  Reverse direction: " + swCrossBreakFeatureData.ReverseDirection);
                    Debug.Print("  Radius: " + swCrossBreakFeatureData.BreakRadius);
                    Debug.Print("  Angle: " + swCrossBreakFeatureData.BreakAngle);
                    swCrossBreakFeatureData.ReleaseSelectionAccess();
                }
            }

            /// <summary>
            /// The SldWorks swApp variable is pre-assigned for you.
            /// </summary>
            public SldWorks swApp;
        }
    }
}
```

## Get Cross Break Feature Data in Sheet Metal Part Example (VBA)

This example shows how to get cross break feature data in a sheet metal part.

```

'-----
' Preconditions:
' 1. Verify that the specified part document template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a sheet metal part with a cross break feature.
' 2. Iterates over all features in the part and gets the
'   cross break feature.
' 3. Names the cross break feature's face.
' 4. Prints to the Immediate window this cross break
'   feature data:
'   * Name of the face
'   * Whether its direction is reversed
'   * Radius
'   * Angle
' 5. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swPart As SldWorks.PartDoc
Dim swModel As SldWorks.ModelDoc2
Dim swSketchManager As SldWorks.SketchManager
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swCustomBendAllowance As SldWorks.CustomBendAllowance
Dim swFeature As SldWorks.Feature
Dim swCrossBreakFeatureData As SldWorks.CrossBreakFeatureData
Dim swFace As SldWorks.Face2
Dim swEntity As SldWorks.Entity
Dim sketchLines As Variant
Dim features As Variant
Dim faceName As String
Dim status As Boolean
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Create sheet metal part with cross break feature
    Set swPart = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModel = swPart
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
    swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
    swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    sketchLines = swSketchManager.CreateCornerRectangle(0, 0, 0, 0.112582864792503, -6.90084337349398E-02, 0)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True
    swModel.ShowNamedView2 "Trimetric", 8
    Set swFeatureManager = swModel.FeatureManager
    Set swCustomBendAllowance = swFeatureManager.CreateCustomBendAllowance()
    swCustomBendAllowance.KFactor = 0.5
    Set swFeature = swFeatureManager.InsertSheetMetalBaseFlange2(0.0007366, False, 0.01905, 0.00508, 0.00254, False, 0, 0, 1, swCustomBendAllowance,
    False, 0, 0.0001, 0.0001, 0.5, True, False, True, True)
    status = swModelDocExt.SelectByID("Base-Flange1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("", "FACE", 4.40948432654409E-02, -3.02769643316765E-02, 0, True, 0, Nothing, 0)
    Set swFeature = swFeatureManager.InsertCrossBreak(1.5707963267949, 0.000635)
    swModel.ClearSelection2 True

    'Get the cross break feature
    'by iterating over all features
    features = swFeatureManager.GetFeatures(True)
    For i = LBound(features) To UBound(features)
        Set swFeature = features(i)
        If swFeature.GetTypeName2 = "CrossBreak" Then
            Set swCrossBreakFeatureData = swFeature.GetDefinition()
            swCrossBreakFeatureData.AccessSelections swModel, Nothing
            Set swFace = swCrossBreakFeatureData.Face
            Set swEntity = swFace
            faceName = "CrossBreakFace"
            status = swPart.SetEntityName(swEntity, faceName)
            faceName = swModel.GetEntityName(swEntity)
            Debug.Print "Cross break feature data:"
            Debug.Print "  Name of face: " & faceName
            Debug.Print "  Reverse direction: " & swCrossBreakFeatureData.ReverseDirection
            Debug.Print "  Radius: " & swCrossBreakFeatureData.BreakRadius
            Debug.Print "  Angle: " & swCrossBreakFeatureData.BreakAngle
            swCrossBreakFeatureData.ReleaseSelectionAccess
        End If
    Next
End Sub

```

## Get Cross Break Feature Data in Sheet Metal Part Example (VB.NET)

This example shows how to get cross break feature data in a sheet metal part.

```

-----
Preconditions:
1. Verify that the specified part document template exists.
2. Open the Immediate window.
Postconditions:
1. Creates a sheet metal part with a cross break feature.
2. Iterates over all features in the part and gets the
   cross break feature.
3. Names the cross break feature's face.
4. Prints to the Immediate window this cross break
   feature data:
   * Name of the face
   * Whether its direction is reversed
   * Radius
   * Angle
5. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swPart As PartDoc
        Dim swModel As ModelDoc2
        Dim swSketchManager As SketchManager
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swCustomBendAllowance As CustomBendAllowance
        Dim swFeature As Feature
        Dim swCrossBreakFeatureData As CrossBreakFeatureData
        Dim swFace As Face2
        Dim swEntity As Entity
        Dim sketchLines As Object
        Dim features As Object
        Dim faceName As String
        Dim status As Boolean
        Dim i As Integer

        'Create sheet metal part with cross break feature
        swPart = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Part.prtdot", 0, 0, 0)
        swModel = swPart
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOption
Specified, False)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOp
tionSpecified, True)
        sketchLines = swSketchManager.CreateCornerRectangle(0, 0, 0, 0.112582864792503, -0.0690084337349398, 0)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)
        swModel.ShowNamedView2("Trimetric", 8)
        swFeatureManager = swModel.FeatureManager
        swCustomBendAllowance = swFeatureManager.CreateCustomBendAllowance()
        swCustomBendAllowance.KFactor = 0.5
        swFeature = swFeatureManager.InsertSheetMetalBaseFlange2(0.0007366, False, 0.01905, 0.00508, 0.00254, False, 0, 0, 1, swCustomBendAllowance, False
, 0, 0, 0.0001, 0.0001, 0.5, True, False, True, True)
        status = swModelDocExt.SelectByID("", "FACE", 0.04409948432654409, -0.0302769643316765, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("", "FACE", 0.04409948432654409, -0.0302769643316765, 0, True, 0, Nothing, 0)
        swFeature = swFeatureManager.InsertCrossBreak(1.5707963267949, 0.000635)
        swModel.ClearSelection2(True)

        'Get the cross break feature
        'by iterating over all features
        features = swFeatureManager.GetFeatures(True)
        For i = LBound(features) To UBound(features)
            swFeature = features(i)
            If swFeature.GetTypeName2 = "CrossBreak" Then
                swCrossBreakFeatureData = swFeature.GetDefinition()
                swCrossBreakFeatureData.AccessSelections(swModel, Nothing)
                swFace = swCrossBreakFeatureData.Face
                swEntity = swFace
                faceName = "CrossBreakFace"
                status = swPart.SetEntityName(swEntity, faceName)
                faceName = swModel.GetEntityName(swEntity)
                Debug.Print("Cross break feature data:")
                Debug.Print("  Name of face: " & faceName)
                Debug.Print("  Reverse direction: " & swCrossBreakFeatureData.ReverseDirection)
                Debug.Print("  Radius: " & swCrossBreakFeatureData.BreakRadius)
                Debug.Print("  Angle: " & swCrossBreakFeatureData.BreakAngle)
                swCrossBreakFeatureData.ReleaseSelectionAccess()
            End If
        Next
    End Sub

    '--- <summary>
    '--- The SldWorks swApp variable is pre-assigned for you.
    '--- </summary>
    Public swApp As SldWorks

End Class

```

### Get Crosshatches on the View Example (VBA)

This example shows how to iterate the drawing views and get the crosshatches in the current drawing view.

```
'-----  
'  
' Preconditions: Drawing document is open and contains at least one drawing view.  
'  
' Postconditions: None  
'  
'-----  
  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swDrawing As SldWorks.DrawingDoc  
Dim currentView As SldWorks.View  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swDrawing = swApp.ActiveDoc  
  
' Iterate the drawing views  
Set currentView = swDrawing.GetFirstView  
While Not currentView Is Nothing  
    ' Get the crosshatch count on the view  
    Debug.Print currentView.GetFaceHatchCount  
    If currentView.GetFaceHatchCount = 0 Then  
        Set currentView = currentView.GetNextView  
    Else  
        ' Get the crosshatches on the view  
        Dim hatcharray As Variant  
        hatcharray = currentView.GetFaceHatches  
  
        ' Following FaceHatch properties are accessible to you  
        Debug.Print hatcharray(1).Pattern  
        Debug.Print hatcharray(1).angle  
        Debug.Print hatcharray(1).Color  
        Debug.Print hatcharray(1).Layer  
        Debug.Print hatcharray(1).Scale2  
        Debug.Print hatcharray(1).SolidFill  
  
        Set currentView = currentView.GetNextView  
    End If  
Wend  
End Sub
```

## Get Curve Segments Example (VBA)

This examples shows how to get the curve segments in a reference curve.

```

'-----
' Preconditions:
' 1. Verify that the part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects a face and sketches a spline on that face.
' 3. Selects the sketch of the spline and a face.
' 4. Inserts a projected curve feature and selects it.
' 5. Gets the number of curve segments and the curve segment.
' 6. Examine the Immediate window, FeatureManager design tree, and
'    the graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swRefCurve As SldWorks.ReferenceCurve
Dim swCurve As SldWorks.Curve
Dim swEdge As SldWorks.Edge
Dim swSketch As SldWorks.Sketch
Dim swEntity As SldWorks.Entity
Dim pointArray As Variant
Dim points(10) As Double
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Sub main()

    Set swApp = Application.SldWorks

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Sketch a spline on the selected face
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("", "FACE", -4.99223104334874E-02, 3.96239999998897E-02, 7.38137362270663E-03, False, 0, Nothing, 0)
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
    points(0) = -6.24778997860176E-02
    points(1) = 7.29572078180673E-03
    points(2) = 0
    points(3) = -3.64588790258153E-02
    points(4) = 3.24586288177215E-02
    points(5) = 0
    points(6) = 1.04252377344665E-02
    points(7) = 1.40473535914225E-02
    points(8) = 0
    points(9) = 6.46002912861796E-02
    points(10) = 1.00590221094308E-02
    pointArray = points
    Set swSketchSegment = swSketchManager.CreateSpline2((pointArray), False)
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True

    'Insert projected curve
    status = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", -4.97146993259321E-02, 0, -2.56283866693252E-02, True, 0, Nothing, 0)
    Set swFeature = swModel.InsertProjectedSketch2(1)

    'Select reference curve and get number of reference curve segments
    status = swModelDocExt.SelectByID("Curve1", "REFCURVE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionManager = swModel.SelectionManager
    Set swFeature = swSelectionManager.GetSelectedObject6(1, -1)
    Set swRefCurve = swFeature.GetSpecificFeature2
    Debug.Print "Number of curve segments: " & swRefCurve.GetSegmentCount
    Set swEdge = swRefCurve.GetFirstSegment
    Debug.Print " Feature = " & swFeature.Name

    'Select each edge
    Do While Not swEdge Is Nothing
        Set swEntity = swEdge
        status = swEntity.Select4(True, Nothing)
        Debug.Print "    Type of entity (1 = edge): " & swEntity.GetType
        Set swEdge = swRefCurve.GetNextSegment
    Loop

End Sub

```

## Get Curve Spline Points Example (C++ COM)

This example shows how to get the point coordinates from a selected sketch spline segment.

The methods `ICurve::IGetSplinePts` and `ICurve::IGetBCurveParams` return arrays, so this code must be used in an in-process DLL. Otherwise, use the methods `ICurve::GetSplinePts` and `ICurve::GetBCurveParams` that return `VARIANTS`.

```
void GetCurveSplinePoints(ISldWorks* m_pSldWorks)
{
    HRESULT hres;

    LPMODELDOC2 pModelDoc = NULL;
    hres = m_pSldWorks->get_IActiveDoc2(&pModelDoc);
    if( pModelDoc == NULL)
        return;

    LPUNKNOWN pUnk = NULL;
    LPSELECTIONMGR pSelectMgr = NULL;
    pModelDoc->get_ISelectionManager( &pSelectMgr);
    if( pSelectMgr)
    {
        pSelectMgr->IGetSelectedObject3( 1, &pUnk);
        pSelectMgr->Release();
    }

    LPsketchSEGMENT pSketchSeg = NULL;

    if( pUnk)
    {
        pUnk->QueryInterface( IID_ISketchSegment, (void **)&pSketchSeg);
        pUnk->Release();
        pUnk = NULL;
    }

    LPCURVE pCurve = NULL;
    if( pSketchSeg)
    {
        pSketchSeg->IGetCurve( &pCurve);
        pSketchSeg->Release();
    }

    if( pCurve)
    {
        long nParamSize;
        pCurve->IGetBCurveParamsSize2( FALSE, TRUE, &nParamSize);
        double *dSplineParams = new double [nParamSize];

        pCurve->IGetBCurveParams( dSplineParams);
        int *propArray = (int*)&dSplineParams[0];

        double *dKnotArray = &dSplineParams[2];
        double *dCtrlArray = &dSplineParams[ 2+ propArray[1]+propArray[2] ];

        long nSplinePts;
        pCurve->IGetSplinePtsSize( propArray, dKnotArray, dCtrlArray, &nSplinePts);
        double *dSplinePts = new double[nSplinePts];

        pCurve->IGetSplinePts( dSplinePts);

        for( int i = 0; i < nSplinePts; i += 3)
        {
            TRACE( auT("x = %f, y = %f, z = %f\n"),

                dSplinePts[i], dSplinePts[i+1], dSplinePts[i+2]);

        }

        delete [] dSplineParams;
        delete [] dSplinePts;
    }
}
```

```
pCurve->Release();  
}  
pModelDoc->Release();  
}
```

## Get Curve Spline Points Example (VBA)

Given a spline curve, this example shows how to get the set of interpolation points.

```
'-----
' Preconditions:
' 1. Open a part containing a sketch of a spline.
' 2. Select the spline.
' 3. Open the Immediate window.
' 4. Run the Curve_GetSplinePts1() subroutine.
'
' Postconditions:
' 1. Gets the parameters and spline points for the selected spline.
' 2. Examine the Immediate window.
' 3. Run Curve_GetSplinePts2() subroutine.
' 4. Gets the parameters and spline points for the selected spline.
' 5. Examine the Immediate window.
'-----
Option Explicit

' Extract two integer values out of a single
' double value, assuming that a C int has 4 bytes
Type DoubleRec
    dValue As Double
End Type

Type Int2Rec
    iLower As Long
    iUpper As Long
End Type

' Extract two integer values out of a single
' double value, by assigning a DoubleRec to the
' double value then copying the value over an
' Int2Rec and extracting the integer values
Function ExtractFields(dValue As Double, iLower As Integer, iUpper As Integer)
    Dim dr As DoubleRec
    Dim i2r As Int2Rec

    ' Set the double value
    dr.dValue = dValue
    ' Copy the values
    LSet i2r = dr
    ' Extract the values
    iLower = i2r.iLower
    iUpper = i2r.iUpper
End Function

Sub DumpSplineInfo(varSplinePts As Variant)
    Dim i As Long
    For i = 0 To UBound(varSplinePts) Step 3
        Debug.Print "    SplinePt(" & i / 3 & ")    = (" & varSplinePts(i + 0) * 1000# & ", " & varSplinePts(i + 1) * 1000# & ", " & varSplinePts(i + 2) *
1000# & ") mm"
    Next i
End Sub

Sub Curve_GetSplinePts1()

    Dim swApp As SldWorks.SldWorks
    Dim Part As SldWorks.ModelDoc2

    Set swApp = CreateObject("SldWorks.Application")
    Set Part = swApp.ActiveDoc
    Dim swSelectMgr As SldWorks.SelectionMgr
    Set swSelectMgr = Part.SelectionManager
    Dim swSketchSeg As SldWorks.SketchSegment
    Set swSketchSeg = swSelectMgr.GetSelectedObject5(1)
    Dim swCurveIn As SldWorks.Curve
    Set swCurveIn = swSketchSeg.GetCurve
    Dim varSplineParams As Variant

    ' False - do not want a cubic spline
    varSplineParams = swCurveIn.GetBCurveParams(False)

    Dim iNumKnots As Integer
    Dim iNumCtrlPts As Integer
    Dim iDimension As Integer
    Dim iOrder As Integer
    Dim iPeriodicity As Integer
    Dim iSplineArraySize As Integer
    Dim dTmpValue1 As Double
    Dim dTmpValue2 As Double
    Dim iSplineIndex As Integer
    Dim iVarIndex As Integer

    dTmpValue1 = varSplineParams(0)
    ExtractFields dTmpValue1, iDimension, iOrder
    dTmpValue2 = varSplineParams(1)
    ExtractFields dTmpValue2, iNumCtrlPts, iPeriodicity
    iNumKnots = iOrder + iNumCtrlPts
    iSplineArraySize = 2 + iNumKnots + (iDimension * iNumCtrlPts)
    Dim dSplineParams() As Double
    ReDim dSplineParams(iSplineArraySize - 1)

    ' Set Property Array
    dSplineParams(0) = dTmpValue1
    dSplineParams(1) = dTmpValue2
```



```

' Set Knot Vector
iSplineIndex = 2
iVarIndex = 2
Dim i As Long
For i = 0 To (iNumKnots - 1)
    dSplineParams(iSplineIndex) = varSplineParams(iVarIndex)
    iSplineIndex = iSplineIndex + 1
    iVarIndex = iVarIndex + 1
Next i

' Set Control Point Vector
For i = 0 To (iNumCtrlPts - 1)
    Dim j As Long
    For j = 1 To iDimension
        dSplineParams(iSplineIndex) = varSplineParams(iVarIndex)
        iSplineIndex = iSplineIndex + 1
        iVarIndex = iVarIndex + 1
    Next j
Next i

Dim varSplinePtParams As Variant
varSplinePtParams = dSplineParams
Dim varSplinePts As Variant
varSplinePts = swCurveIn.GetSplinePts((varSplinePtParams))

DumpSplineInfo varSplinePts

End Sub

Sub Curve_GetSplinePts2()
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2

Set swApp = CreateObject("SldWorks.Application")
Set Part = swApp.ActiveDoc
Dim swSelectMgr As SldWorks.SelectionMgr
Set swSelectMgr = Part.SelectionManager
Dim swSketchSeg As SldWorks.SketchSegment
Set swSketchSeg = swSelectMgr.GetSelectedObject3(1)
Dim swCurveIn As SldWorks.Curve
Set swCurveIn = swSketchSeg.GetCurve
Dim varSplineParams As Variant

' False - do not want a cubic spline
varSplineParams = swCurveIn.GetBCurveParams(False)
Dim varSplinePts As Variant
varSplinePts = swCurveIn.GetSplinePts((varSplineParams))

DumpSplineInfo varSplinePts

End Sub

```

## Get Curves that Form Outline of Bodies Example (VBA)

This example shows how to get the curves that form the outline of a body and create a 3D sketch using those curves.

```

'-----
' Preconditions:
' 1. Copy and paste Main into your macro.
' 2. Click Insert > Class Module and copy and paste OutlineCurve
'    in the class module.
'    a. Click View > Project Explorer.
'    b. Click View > Properties Window.
'    c. Type OutlineCurve in (Name) in the Properties window.
' 3. Click Insert > Class Module and copy and paste Outline in
'    the class module.
'    a. Click View > Properties Window.
'    b. Type Outline in (Name) in the Properties window.
' 4. Add a reference to the Microsoft Scripting Runtime (click
'    Tools > References > Microsoft Scripting Runtime > OK).
' 5. Open a part document containing at least one body.
'
' Postconditions:
' 1. Creates a 3D sketch using the outlines of the curves
'    in the bodies.
' 2. Examine the graphics area, FeatureManager design tree,
'    and the graphics area.
'-----
'Main
Option Explicit

Dim swApp As SldWorks.SldWorks

Sub main()

    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim swModeler As SldWorks.Modeler
    Dim swMathUtility As SldWorks.MathUtility
    Dim vBodies As Variant
    Dim swBody As SldWorks.Body2
    Dim aVector(2) As Double
    Dim vVector As Variant
    Dim swVector As SldWorks.MathVector
    Dim dTolerance As Double
    Dim vCurves As Variant
    Dim vTopologicalEntities As Variant
    Dim vIndices As Variant
    Dim lNumCurves As Long
    Dim oOutlineCurve As OutlineCurve
    Dim oOutline As Outline
    Dim dictOutLines As Dictionary
    Dim lOutlineIdx As Long
    Dim lIdx As Long
    Dim vOutline As Variant
    Dim vOutlineCurve As Variant
    Dim swEntity As SldWorks.Entity
    Dim swEdge As SldWorks.Edge
    Dim swFace As SldWorks.Face2
    Dim bValue As Boolean
    Dim swCurve As SldWorks.Curve
    Dim dStartParam As Double
    Dim dEndParam As Double
    Dim bIsClosed As Boolean
    Dim bIsPeriodic As Boolean
    Dim vStartPoint As Variant
    Dim vEndPoint As Variant
    Dim vCircleParams As Variant
    Dim aCenterPoint(2) As Double
    Dim vTessPts As Variant
    Dim nChordTol As Double
    Dim nLengthTol As Double
    Dim swSketchSegment As SldWorks.SketchSegment
    Dim swSketchManager As SldWorks.SketchManager
    Dim aColours(5) As Long
    Dim swObject As Object
    Dim nbrTessPoints As Long

    nChordTol = 0.00000001
    nLengthTol = 0.0000000000001
    aColours(0) = RGB(255, 0, 0)
    aColours(1) = RGB(0, 255, 0)
    aColours(2) = RGB(0, 0, 255)
    aColours(3) = RGB(255, 255, 0)
    aColours(4) = RGB(255, 0, 255)
    aColours(5) = RGB(0, 255, 255)

    Set swApp = Application.SldWorks
    Set swModeler = swApp.GetModeler
    Set swMathUtility = swApp.GetMathUtility
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel
    Set swSketchManager = swModel.SketchManager

    vBodies = swPart.GetBodies2(swBodyType_e.swSolidBody, False)

    ' Look along the z-axis in the negative direction;
    ' this corresponds to the Front view

```

```

aVector(0) = 0#
aVector(1) = 0#
aVector(2) = -1#
vVector = aVector
Set swVector = swMathUtility.CreateVector((vVector))

' Default value
dTolerance = 0.00001

lNumCurves = swModeler.GetBodyOutline2((vBodies), swVector, dTolerance, False, vCurves, vTopologicalEntities, vIndices)
If (lNumCurves > 0) Then
    Debug.Print "#curves = " & lNumCurves
    Set dictOutLines = New Dictionary
    lOutlineIdx = -1
    For lIdx = 0 To (lNumCurves - 1)
        If (vIndices(lIdx) <> lOutlineIdx) Then
            lOutlineIdx = vIndices(lIdx)
            Set oOutline = New Outline
            oOutline.lIndex = lOutlineIdx
            dictOutLines.Add lOutlineIdx, oOutline
        End If
        Set oOutlineCurve = New OutlineCurve
        Set oOutlineCurve.swCurve = vCurves(lIdx)
        Set oOutlineCurve.swEntity = vTopologicalEntities(lIdx)
        Set swObject = vTopologicalEntities(lIdx)
        If (TypeOf swObject Is SldWorks.Edge) Then
            ' HERE: real edge
            oOutlineCurve.nType = swSelEDGES
        End If
        Set swObject = vTopologicalEntities(lIdx)
        If (TypeOf swObject Is SldWorks.Face2) Then
            ' HERE: silhouette edge
            oOutlineCurve.nType = swSelFACES
        End If
        oOutline.dictCurves.Add oOutlineCurve, oOutlineCurve
    Next lIdx

    swModel.SetAddToDB True
    swModel.SetDisplayWhenAdded False
    swModel.Insert3DSketch2 False

    For Each vOutline In dictOutLines.Items
        swModel.ClearSelection2 True
        Set oOutline = vOutline
        Debug.Print "Outline " & oOutline.lIndex
        Debug.Print " #curves = " & oOutline.dictCurves.Count
        For Each vOutlineCurve In oOutline.dictCurves.Items
            Set oOutlineCurve = vOutlineCurve
            Set swCurve = oOutlineCurve.swCurve
            Debug.Print " type = " & swCurve.Identity
            Debug.Print " trimmed = " & swCurve.IsTrimmedCurve
            Set swEntity = oOutlineCurve.swEntity
            bValue = swEntity.Select4(True, Nothing)

            ' Draw some of the curves to show where they live in 3D space
            bValue = swCurve.GetEndParams(dStartParam, dEndParam, bIsClosed, bIsPeriodic)
            vStartPoint = swCurve.Evaluate(dStartParam)
            vEndPoint = swCurve.Evaluate(dEndParam)
            vTessPts = swCurve.GetTessPts(nChordTol, nLengthTol, (vStartPoint), (vEndPoint))
            For lIdx = 0 To nbrTessPoints Step 3
                Set swSketchSegment = swSketchManager.CreateLine(vTessPts(lIdx + 0), vTessPts(lIdx + 1), vTessPts(lIdx + 2), vTessPts(lIdx + 3),
vTessPts(lIdx + 4), vTessPts(lIdx + 5))
                swSketchSegment.Color = aColours((oOutline.lIndex Mod UBound(aColours)))
            Next lIdx
        Next vOutlineCurve
    Next vOutline

    swModel.Insert3DSketch2 True
    swModel.SetAddToDB False
    swModel.SetDisplayWhenAdded True
End If

End Sub

'OutlineCurve
Option Explicit
Public swCurve As SldWorks.Curve
Public nType As SwConst.swSelectType_e
Public swEntity As SldWorks.Entity
Private Sub Class_Initialize()
    Set swCurve = Nothing
    nType = SwConst.swSelectType_e.swSelNOTHING
    Set swEntity = Nothing
End Sub

'Outline
Option Explicit
Public lIndex As Long
Public dictCurves As Dictionary
Private Sub Class_Initialize()
    lIndex = -1
    Set dictCurves = New Dictionary
End Sub

```

### Get Custom Properties Example (VBA)

This example shows how to get the custom properties for the configurations in a document.

```
'-----
Option Explicit

Public Enum swCustomInfoType_e
    swCustomInfoUnknown = 0
    swCustomInfoText = 30    ' VT_LPSTR
    swCustomInfoDate = 64    ' VT_FILETIME
    swCustomInfoNumber = 3    ' VT_I4
    swCustomInfoYesOrNo = 11 ' VT_BOOL
End Enum

Sub main()

    Dim swApp          As SldWorks.SldWorks
    Dim swModel         As SldWorks.ModelDoc2
    Dim swConfig        As SldWorks.Configuration
    Dim vConfName       As Variant
    Dim vPropName       As Variant
    Dim vPropValue      As Variant
    Dim vPropType       As Variant
    Dim nNumProp        As Long
    Dim i               As Long
    Dim j               As Long
    Dim bRet            As Boolean
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Debug.Print "File = " + swModel.GetPathName

    vConfName = swModel.GetConfigurationNames
    For i = 0 To UBound(vConfName)
        Set swConfig = swModel.GetConfigurationByName(vConfName(i))

        nNumProp = swConfig.GetCustomProperties(vPropName, vPropValue, vPropType)

        Debug.Print " Config    = " & vConfName(i)
        For j = 0 To nNumProp - 1
            Debug.Print "   " & vPropName(j) & " <" & vPropType(j) & "> = " & vPropValue(j)
        Next j
        Debug.Print " -----"
    Next i
End Sub
```

## Get Custom Properties for BOM Table Columns Example (VBA)

This example shows how to get the custom properties used to fill columns in a BOM table.

```

'-----
' Preconditions:
' 1. Specified drawing and BOM table template exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts an indented BOM table.
' 3. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save any changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim swBOMTableAnnotation As SldWorks.BomTableAnnotation
Dim swTableAnnotation As SldWorks.TableAnnotation
Dim status As Boolean
Dim anchorType As Long
Dim bomType As Long
Dim errors As Long
Dim warnings As Long
Dim tableTemplate As String
Dim config As String
Dim i As Long

Option Explicit

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw", swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawing = swModel
    status = swDrawing.ActivateView("Drawing View1")
    Set swView = swDrawing.ActiveDrawingView

    ' Insert indented BOM table and get custom properties for each column in BOM table
    anchorType = swBOMConfigurationAnchor_TopLeft
    bomType = swBomType_Indented
    tableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
    config = ""
    Set swBOMTableAnnotation = swView.InsertBomTable4(False, 0.4, 0.3, anchorType, bomType, config, tableTemplate, False, swNumberingType_Detailed, True)
    Set swTableAnnotation = swBOMTableAnnotation
    For i = 0 To swTableAnnotation.ColumnCount
        Debug.Print " Column(" & i & ") = " & swBOMTableAnnotation.GetColumnCustomProperty(i)
    Next i

    swDrawing.ForceRebuild

End Sub

```

## Get Custom Properties for Configuration Example (C#)

This example shows how to get the names, types, values, and evaluated values of the active configuration's custom properties. It also shows how to add a custom property to the configuration.

```
//-----
// Preconditions:
// 1. Open a part document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Adds a date custom property to the part's configuration.
// 2. Tests whether the custom property is editable, and if so,
//    edits it.
// 3. Gets all custom properties in the configuration.
// 4. Deletes a custom property.
// 5. Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;

namespace IsCustomPropEditable_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            Configuration config;
            CustomPropertyManager cusPropMgr;
            int lRetVal;
            object vPropNames = null;
            string[] propNames;
            object vPropTypes = null;
            object vPropValues = null;
            object[] propValues;
            string ValOut;
            string ResolvedValOut;
            bool wasResolved;
            bool linkToProp;
            object resolved = null;
            object linkProp = null;
            int nNbrProps;
            int j;
            int custPropType;

            bool bRet;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            config = (Configuration)swModel.GetActiveConfiguration();

            cusPropMgr = config.CustomPropertyManager;

            lRetVal = cusPropMgr.Add3("ADATE", (int)swCustomInfoType_e.swCustomInfoDate, "4-13-
59", (int)swCustomPropertyAddOption_e.swCustomPropertyDeleteAndAdd);
            lRetVal = cusPropMgr.Get6("ADATE", false, out ValOut, out ResolvedValOut, out wasResolved, out linkToProp);
            bRet = cusPropMgr.IsCustomPropertyEditable("ADATE", config.Name);
            if (bRet == true)
            {
                Debug.Print("ADATE is editable. Change the date.");
                lRetVal = cusPropMgr.Set2("ADATE", "4-13-89");
            }
            else
            {
                Debug.Print("ADATE is not editable.");
            }

            lRetVal = cusPropMgr.Get6("ADATE", false, out ValOut, out ResolvedValOut, out wasResolved, out linkToProp);

            // Get the number of custom properties for this configuration
            nNbrProps = cusPropMgr.Count;
            Debug.Print("Number of custom properties for this configuration: " + nNbrProps);

            // Get the custom properties
            lRetVal = cusPropMgr.GetAll3(ref vPropNames, ref vPropTypes, ref vPropValues, ref resolved, ref linkProp);
            propValues = (object[])vPropValues;
            propNames = (string[])vPropNames;
            // For each custom property, print its name, type, and evaluated value
            for (j = 0; j <= nNbrProps - 1; j++)
            {
                custPropType = cusPropMgr.GetType2(propNames[j]);
                Debug.Print("Name, swCustomInfoType_e value, and resolved value: " + propNames[j] + ", " + custPropType + ", " + pr
            }

            lRetVal = cusPropMgr.Delete2("ADATE");
        }
    }
}
```

```
        // Get the number of custom properties for this configuration
        nNbrProps = cusPropMgr.Count;
        Debug.Print("Number of custom properties for this configuration: " + nNbrProps);
    }

    // The SldWorks swApp variable is pre-assigned for you.
    public SldWorks swApp;
}
}
```

## Get Custom Properties for Configuration Example (VBA)

This example shows how to get the names, types, and evaluated values of the custom properties in the active configuration. It also shows how to add a custom property to the configuration.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Adds a date custom property to the part's configuration.
' 2. Tests whether the custom property is editable, and if so,
'    edits it.
' 3. Gets all custom properties in the configuration.
' 4. Deletes a custom property.
' 5. Examine the Immediate window.
'-----

```

## Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim config As SldWorks.Configuration
Dim cusPropMgr As SldWorks.CustomPropertyManager
Dim lRetVal As Long
Dim vPropNames As Variant
Dim vPropTypes As Variant
Dim vPropValues As Variant
Dim ValOut As String
Dim ResolvedValOut As String
Dim wasResolved As Boolean
Dim linkToProp As Boolean
Dim resolved As Variant
Dim linkProp As Variant
Dim nNbrProps As Long
Dim j As Long
Dim custPropType As Long
Dim bRet As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set config = swModel.GetActiveConfiguration

    Set cusPropMgr = config.CustomPropertyManager

    lRetVal = cusPropMgr.Add3("ADATE", swCustomInfoType_e.swCustomInfoDate, "4-13-59",
swCustomPropertyAddOption_e.swCustomPropertyDeleteAndAdd)
    lRetVal = cusPropMgr.Get6("ADATE", False, ValOut, ResolvedValOut, wasResolved, linkToProp)

    bRet = cusPropMgr.IsCustomPropertyEditable("ADATE", config.Name)
    If bRet = True Then
        Debug.Print "    ADATE is editable."
        lRetVal = cusPropMgr.Set2("ADATE", "4-13-89")
    Else
        Debug.Print "    ADATE is not editable."
    End If

    lRetVal = cusPropMgr.Get6("ADATE", False, ValOut, ResolvedValOut, wasResolved, linkToProp)

    ' Get the number of custom properties for this configuration
    nNbrProps = cusPropMgr.Count
    Debug.Print "Number of properties for this configuration: " & nNbrProps

    ' Gets the custom properties
    lRetVal = cusPropMgr.GetAll3(vPropNames, vPropTypes, vPropValues, resolved, linkProp)

    ' For each custom property, print its name, type, and evaluated value
    For j = 0 To nNbrProps - 1
        custPropType = cusPropMgr.GetType2(vPropNames(j))
        Debug.Print "    Name, swCustomInfoType_e value, and resolved value: " & vPropNames(j) & ", "; custPropType & ", " &
vPropValues(j)
    Next j

    lRetVal = cusPropMgr.Delete2("ADATE")

    ' Get the number of custom properties for this configuration
    nNbrProps = cusPropMgr.Count
    Debug.Print "Number of properties for this configuration: " & nNbrProps
End Sub

```



## Get Custom Properties for Configuration Example (VB.NET)

This example shows how to get the names, types, values, and evaluated values of the active configuration's custom properties. It also shows how to add a custom property to the configuration.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Adds a date custom property to the part's configuration.
' 2. Tests whether the custom property is editable, and if so,
'    edits it.
' 3. Gets all custom properties in the configuration.
' 4. Deletes a custom property.
' 5. Examine the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

Partial Class SolidWorksMacro

```

    Dim swModel As ModelDoc2
    Dim config As Configuration
    Dim cusPropMgr As CustomPropertyManager
    Dim lRetVal As Integer
    Dim vPropNames As Object
    Dim vPropTypes As Object
    Dim vPropValues As Object
    Dim ValOut As String
    Dim ResolvedValOut As String
    Dim wasResolved As Boolean
    Dim linkToProp As Boolean
    Dim resolved As Object
    Dim linkProp As Object
    Dim nNbrProps As Integer
    Dim j As Integer
    Dim custPropType As Integer
    Dim bRet As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        config = swModel.GetActiveConfiguration

        cusPropMgr = config.CustomPropertyManager

        lRetVal = cusPropMgr.Add3("ADATE", swCustomInfoType_e.swCustomInfoDate, "4-13-
59", swCustomPropertyAddOption_e.swCustomPropertyDeleteAndAdd)
        lRetVal = cusPropMgr.Get6("ADATE", False, ValOut, ResolvedValOut, wasResolved, linkToProp)
        bRet = cusPropMgr.IsCustomPropertyEditable("ADATE", config.Name)
        If bRet = True Then
            Debug.Print("ADATE is editable. Change the date.")
            lRetVal = cusPropMgr.Set2("ADATE", "4-13-89")
        Else
            Debug.Print("ADATE is not editable.")
        End If

        lRetVal = cusPropMgr.Get6("ADATE", False, ValOut, ResolvedValOut, wasResolved, linkToProp)

        ' Get the number of custom properties for this configuration
        nNbrProps = cusPropMgr.Count
        Debug.Print("Number of custom properties for this configuration: " & nNbrProps)

        ' Get the custom properties
        lRetVal = cusPropMgr.GetAll3(vPropNames, vPropTypes, vPropValues, resolved, linkProp)

        ' For each custom property, print its name, type, and evaluated value
        For j = 0 To nNbrProps - 1
            custPropType = cusPropMgr.GetType2(vPropNames(j))
            Debug.Print("Name, swCustomInfoType_e value, and resolved value: " & vPropNames(j) & ", " & custPropType & ", " & vProp
        Next j

        lRetVal = cusPropMgr.Delete2("ADATE")

        ' Get the number of custom properties for this configuration
        nNbrProps = cusPropMgr.Count
        Debug.Print("Number of custom properties for this configuration: " & nNbrProps)

    End Sub

```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

```

Public swApp As Sldworks

End Class

## Get Custom Properties for Weldment Cut-list Item Example (VBA)

This example shows how to get the custom properties for the selected weldment cut-list item.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
' 2. Click Tools > Options > Document Properties > Weldments >
'    Rename cut list folders with Description property value > OK.
' 3. Right-click Cut list(31) in the FeatureManager design tree
'    and click Update.
' 4. Expand Cut list(31) and click TUBE, SQUARE 30 X 30 X 2.60<1>.
' 5. Open the Immediate window
'
' Postconditions:
' 1. Gets the custom properties for the selected cut-list item.
' 2. Examine the Immediate window.
'
' NOTE: Because the part document is used elsewhere, do not
'       save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swCutListItem As SldWorks.Feature
    Dim swCustPropMgr As SldWorks.CustomPropertyManager
    Dim names As Variant
    Dim name As Variant
    Dim textexp As String
    Dim evalval As String

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swCutListItem = swSelMgr.GetSelectedObject6(1, 0)

    Set swCustPropMgr = swCutListItem.CustomPropertyManager
    Debug.Print "Custom properties for selected weldment cut-list item"
    Debug.Print "Number of custom properties = " + CStr(swCustPropMgr.Count)
    Debug.Print "Name", "Text Expression", "Value", "Type"

    names = swCustPropMgr.GetNames
    For Each name In names
        swCustPropMgr.Get2 name, textexp, evalval
        Debug.Print name, textexp, evalval, swCustPropMgr.GetType(name)
    Next name

End Sub
```

## Get Custom Properties of Referenced Part Example (C#)

This example shows how to get the custom property data of a referenced part.

```
//-----
// Preconditions:
// 1. Open a part that contains a referenced part
//    with a custom property and a value.
// 2. Open the referenced part.
// 3. Switch back to the part opened in step 1.
// 4. In the macro, replace Property_Name with the name of
//    the referenced part's custom property.
// 5. Open the Immediate window.
//
// Postconditions: Prints the custom property data
// to the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace Get4CustomPropertyManagerCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            CustomPropertyManager swCustProp = default(CustomPropertyManager);
            string val = "";
            string valout = "";
            bool status;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;

            // Get the custom property data
            swCustProp = swModelDocExt.get_CustomPropertyManager("");
            status = swCustProp.Get4("Property_Name", false, out val, out valout);

            Debug.Print("Value:                " + val);
            Debug.Print("Evaluated value:        " + valout);
            Debug.Print("Up-to-date data:        " + status);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

### Get Custom Properties of Referenced Part Example (VBA)

This example shows how to get the custom property data of a referenced part.

```
'-----  
' Preconditions:  
' 1. Open a part that contains a referenced part  
'    with a custom property and a value.  
' 2. Open the referenced part.  
' 3. Switch back to the part opened in step 1.  
' 4. In the macro, replace Property_Name with the name of  
'    the referenced part's custom property.  
' 5. Open the Immediate window.  
'  
' Postconditions: Prints the custom property data  
' to the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As ModelDoc2  
Dim swModelDocExt As ModelDocExtension  
Dim swCustProp As CustomPropertyManager  
Dim val As String  
Dim valout As String  
Dim bool As Boolean  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swModelDocExt = swModel.Extension  
  
' Get the custom property data  
Set swCustProp = swModelDocExt.CustomPropertyManager("")  
bool = swCustProp.Get4("Property_Name", False, val, valout)  
  
Debug.Print "Value:                " & val  
Debug.Print "Evaluated value:          " & valout  
Debug.Print "Up-to-date data:            " & bool  
  
End Sub
```

## Get Custom Properties of Referenced Part Example (VB.NET)

This example shows how to get the custom property data of a referenced part.

```

'-----
' Preconditions:
' 1. Open a part that contains a referenced part
'   with a custom property and a value.
' 2. Open the referenced part.
' 3. Switch back to the part opened in step 1.
' 4. In the macro, replace Property_Name with the name of
'   the referenced part's custom property.
' 5. Open the Immediate window.
'
' Postconditions: Prints the custom property data
' to the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swCustProp As CustomPropertyManager
        Dim val As String = ""
        Dim valout As String = ""
        Dim bool As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Get the custom property data
        swCustProp = swModelDocExt.CustomPropertyManager("")
        bool = swCustProp.Get4("Property_Name", False, val, valout)

        Debug.Print("Value:                " & val)
        Debug.Print("Evaluated value:            " & valout)
        Debug.Print("Up-to-date data:            " & bool)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

End Class

## Get Custom Property Values on Weldment Cut-list Folders Example (VBA)

This example shows how to get all of the custom property values on the weldment cut-list folders of a part in an assembly.

```
'-----
' Preconditions:
' 1. Open a new SOLIDWORKS Design session.
' 2. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
' 3. Click Tools > Options > Document Properties > Weldments >
'    Rename cut list folders with Description property value > OK.
' 4. Right-click Cut list(31) in the FeatureManager design tree
'    and click Update.
' 5. Verify that the specified assembly template exists.
' 6. Open the Immediate window.
'
' Postconditions:
' 1. Creates an assembly document and inserts the part document.
' 2. Traverses the part's FeatureManager design tree and gets
'    the names of custom properties, values, and evaluated values
'    for the cut-list folders in the part document.
' 3. Examine the Immediate window.
'
' NOTE: Because the part document is used elsewhere, do not
' save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssem As SldWorks.AssemblyDoc
Dim swSelMgr As SldWorks.SelectionMgr
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
```

```
Sub VisitFeatureCustomProperties(swDocFeat As SldWorks.Feature)
    Dim swCustPropMgr As SldWorks.CustomPropertyManager
    Dim propNames As Variant
    Dim vName As Variant
    Dim propName As String
    Dim Value As String
    Dim resolvedValue As String

    Set swCustPropMgr = swDocFeat.CustomPropertyManager
    If Not swCustPropMgr Is Nothing Then
        propNames = swCustPropMgr.GetNames
        If Not IsEmpty(propNames) Then
            Debug.Print swDocFeat.Name, swDocFeat.GetTypeName
            For Each vName In propNames
                propName = vName
                Call swCustPropMgr.Get2(propName, Value, resolvedValue)
                Debug.Print "", "", propName, Value, resolvedValue
            Next vName
        End If
    End If
End Sub
```

```
Sub VisitDocWeldmentProperties(swCompDoc As SldWorks.ModelDoc2)
    Dim swFeature As SldWorks.Feature
    Dim swSubFeature As SldWorks.Feature
    Dim swCutFolder As SldWorks.BodyFolder

    Set swFeature = swCompDoc.FirstFeature
    Do While Not swFeature Is Nothing
        Set swSubFeature = swFeature.GetFirstSubFeature
        Do While Not swSubFeature Is Nothing
            If swSubFeature.GetTypeName2 = "CutListFolder" Then
                Set swCutFolder = swSubFeature.GetSpecificFeature2
            End If
            If Not swCutFolder Is Nothing Then
                Call VisitFeatureCustomProperties(swSubFeature)
            End If
            Set swSubFeature = swSubFeature.GetNextSubFeature
        Loop
        Set swFeature = swFeature.GetNextFeature
    Loop
End Sub
```

```
Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\assembly.asmdot", 0, 0, 0)
    swApp.ActivateDoc2 "Assem1", False, errors
    Set swModel = swApp.ActiveDoc
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\weldment_box3.sldprt"
    Set swAssem = swModel
    swApp.ActivateDoc2 "Assem1", False, errors
    swAssem.AddComponent5 fileName, swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "", 0.508489013092717,
    0.724898979334123, 0.550645508621615
```

```

Set swSelMgr = swModel.SelectionManager
Set swModelDocExt = swModel.Extension
status = swModelDocExt.SelectByID2("weldment_box3-1@Assem1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)

Dim swSelComp As SldWorks.Component2
Dim refConfig As String
Dim swCompDoc As SldWorks.ModelDoc2

Set swSelComp = swSelMgr.GetSelectedObject6(1, -1)
Set swCompDoc = swSelComp.GetModelDoc

Dim configNames As Variant
Dim vName As Variant
Dim configName As String
configNames = swCompDoc.GetConfigurationNames()
For Each vName In configNames
    configName = vName
    Debug.Print "-----"
    Debug.Print "Configuration: " + configName
    status = swCompDoc.ShowConfiguration2(configName)
    Call VisitDocWeldmentProperties(swCompDoc)
Next vName

End Sub

```



## Get Data for Scale Feature Example (C#)

This example shows how to get the data for the selected scale feature.

```
//-----
// Preconditions:
// 1. Open a model document with a scale feature.
// 2. Select the scale feature.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace ScaleFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            ScaleFeatureData swScale = default(ScaleFeatureData);
            Feature swCoordSys = default(Feature);
            double nX_Scale = 0;
            double nY_Scale = 0;
            double nZ_scale = 0;
            bool bIsUniform = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swScale = (ScaleFeatureData)swFeat.GetDefinition();
            swCoordSys = (Feature)swScale.CoordinateSystem;

            swScale.GetScale(out nX_Scale, out nY_Scale, out nZ_scale, out bIsUniform);

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("Feature = " + swFeat.Name);
            Debug.Print("  X scaling factor = " + nX_Scale);
            Debug.Print("  Y scaling factor = " + nY_Scale);
            Debug.Print("  Z scaling factor = " + nZ_scale);
            Debug.Print("  Is uniform = " + bIsUniform);
            Debug.Print("  Scaling type as defined in swScaleType_e = " + swScale.Type);
            Debug.Print("  Bodies count = " + swScale.GetBodiesCount());
            Debug.Print("  Is uniform = " + swScale.IsUniform);
            Debug.Print("  Uniform scaling factor = " + swScale.ScaleUniform);
            Debug.Print("  X scaling factor = " + swScale.ScaleX);
            Debug.Print("  Y scaling factor = " + swScale.ScaleY);
            Debug.Print("  Z scaling factor = " + swScale.ScaleZ);

            if ((swCoordSys != null))
            {
                Debug.Print("  CoordSys = " + swCoordSys.Name);
            }
        }

        public SldWorks swApp;
    }
}
}
```

## Get Data for Scale Feature Example (VBA)

This example shows how to get the data for the selected scale feature.

```
'-----
' Preconditions:
' 1. Open a model document with a scale feature.
' 2. Select the scale feature.
'
' Postconditions: Inspect the Immediate window.
'-----
```

Option Explicit

Sub main()

```
    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swSelMgr              As SldWorks.SelectionMgr
    Dim swFeat               As SldWorks.Feature
    Dim swScale              As SldWorks.ScaleFeatureData
    Dim swCoordSys            As SldWorks.Feature
    Dim nX_Scale             As Double
    Dim nY_Scale             As Double
    Dim nZ_scale             As Double
    Dim bIsUniform           As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swScale = swFeat.GetDefinition
    Set swCoordSys = swScale.CoordinateSystem

    swScale.GetScale nX_Scale, nY_Scale, nZ_scale, bIsUniform

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print "Feature = " & swFeat.Name
    Debug.Print "  X scaling factor = " & nX_Scale
    Debug.Print "  Y scaling factor = " & nY_Scale
    Debug.Print "  Z scaling factor = " & nZ_scale
    Debug.Print "  Is uniform = " & bIsUniform
    Debug.Print "  Scaling type as defined in swScaleType_e = " & swScale.Type
    Debug.Print "  Bodies count = " & swScale.GetBodiesCount
    Debug.Print "  Is uniform = " & swScale.IsUniform
    Debug.Print "  Uniform scaling factor = " & swScale.ScaleUniform
    Debug.Print "  X scaling factor = " & swScale.ScaleX
    Debug.Print "  Y scaling factor = " & swScale.ScaleY
    Debug.Print "  Z scaling factor = " & swScale.ScaleZ

    If Not swCoordSys Is Nothing Then
        Debug.Print "  CoordSys = " & swCoordSys.Name
    End If
End Sub
```

End Sub

## Get Data for Scale Feature Example (VB.NET)

This example shows how to get the data for the selected scale feature.

```

'-----
' Preconditions:
' 1. Open a model document with a scale feature.
' 2. Select the scale feature.
'-----
' Postconditions: Inspect the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swScale As ScaleFeatureData
        Dim swCoordSys As Feature
        Dim nX_Scale As Double
        Dim nY_Scale As Double
        Dim nZ_scale As Double
        Dim bIsUniform As Boolean

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swScale = swFeat.GetDefinition
        swCoordSys = swScale.CoordinateSystem

        swScale.GetScale(nX_Scale, nY_Scale, nZ_scale, bIsUniform)

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print("Feature = " & swFeat.Name)
        Debug.Print("  X scaling factor = " & nX_Scale)
        Debug.Print("  Y scaling factor = " & nY_Scale)
        Debug.Print("  Z scaling factor = " & nZ_scale)
        Debug.Print("  Is uniform = " & bIsUniform)
        Debug.Print("  Scaling type as defined in swScaleType_e = " & swScale.Type)
        Debug.Print("  Bodies count = " & swScale.GetBodiesCount)
        Debug.Print("  Is uniform = " & swScale.IsUniform)
        Debug.Print("  Uniform scaling factor = " & swScale.ScaleUniform)
        Debug.Print("  X scaling factor = " & swScale.ScaleX)
        Debug.Print("  Y scaling factor = " & swScale.ScaleY)
        Debug.Print("  Z scaling factor = " & swScale.ScaleZ)

        If Not swCoordSys Is Nothing Then
            Debug.Print("  CoordSys = " & swCoordSys.Name)
        End If

    End Sub

    Public swApp As SldWorks

End Class

```

## Get Data for Fillet Feature Example (C#)

This example shows how to get the data for the selected fillet feature.

```
//-----
// Preconditions:
// 1. Open a model document with a simple fillet feature.
// 2. Select the simple fillet feature.
// 3. Open the Immediate window.
// 4. Run the macro.
//
// Postconditions: Inspect the Immediate window.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace SimpleFilletFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            SimpleFilletFeatureData2 swFillet = default(SimpleFilletFeatureData2);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swFillet = (SimpleFilletFeatureData2)swFeat.GetDefinition();

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("Feature = " + swFeat.Name);
            Debug.Print("    Constant width? " + swFillet.ConstantWidth);
            Debug.Print("    Curvature continuous? " + swFillet.CurvatureContinuous);
            Debug.Print("    Default radius = " + swFillet.DefaultRadius);
            Debug.Print("    Number of fillet items = " + swFillet.FilletItemsCount);
            Debug.Print("    Multiple radii? " + swFillet.IsMultipleRadius);
            Debug.Print("    Keep existing features? " + swFillet.KeepFeatures);
            Debug.Print("    Apply fillet to attachment edges? " + swFillet.OmitAttachedEdges);
            Debug.Print("    Overflow type = " + swFillet.OverflowType);
            Debug.Print("    Extend fillet to all affected parts? " + swFillet.PropagateFeatureToParts);
            Debug.Print("    Extend fillet to all tangent faces? " + swFillet.PropagateToTangentFaces);
            Debug.Print("    Reverse normal? " + swFillet.get_ReverseFaceNormal(0));
            Debug.Print("    Round fillet corners? " + swFillet.RoundCorners);
            Debug.Print("    Trim and attach to surfaces? " + swFillet.TrimAndAttachSurfaces);
            Debug.Print("    Type of fillet? " + swFillet.Type);
            Debug.Print("    Number of faces associated with this fillet = " +
            swFillet.GetFaceCount((int)swSimpleFilletWhichFaces_e.swSimpleFilletSingleRadius));

        }

        public SldWorks swApp;
    }
}
```

## Get Data for Fillet Feature Example (VBA)

This example shows how to get the data for the selected fillet feature.

```
'-----
' Preconditions:
' 1. Open a model document with a simple fillet feature.
' 2. Select the simple fillet feature.
' 3. Open the Immediate window.
' 4. Run the macro.
'
' Postconditions: Inspect the Immediate window.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swFillet As SldWorks.SimpleFilletFeatureData2
```

```
Set swApp = CreateObject("SldWorks.Application")
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
Set swFillet = swFeat.GetDefinition
```

```
Debug.Print "File = " & swModel.GetPathName
Debug.Print "Feature = " & swFeat.Name
Debug.Print "    Constant width? " & swFillet.ConstantWidth
Debug.Print "    Curvature continuous? " & swFillet.CurvatureContinuous
Debug.Print "    Default radius = " & swFillet.DefaultRadius
Debug.Print "    Number of fillet items = " & swFillet.FilletItemsCount
Debug.Print "    Multiple radii? " & swFillet.IsMultipleRadius
Debug.Print "    Keep existing features? " & swFillet.KeepFeatures
Debug.Print "    Apply fillet to attachment edges? " & swFillet.OmitAttachedEdges
Debug.Print "    Overflow type = " & swFillet.OverflowType
Debug.Print "    Extend fillet to all affected parts? " & swFillet.PropagateFeatureToParts
Debug.Print "    Extend fillet to all tangent faces? " & swFillet.PropagateToTangentFaces
Debug.Print "    Reverse normal? " & swFillet.ReverseFaceNormal(0)
Debug.Print "    Round fillet corners? " & swFillet.RoundCorners
Debug.Print "    Trim and attach to surfaces? " & swFillet.TrimAndAttachSurfaces
Debug.Print "    Type of fillet? " & swFillet.Type
Debug.Print "    Number of faces associated with this fillet = " &
```

```
swFillet.GetFaceCount(swSimpleFilletWhichFaces_e.swSimpleFilletSingleRadius)
```

End Sub

## Get Data for Fillet Feature Example (VB.NET)

This example shows how to get the data for the selected fillet feature.

```

'-----
' Preconditions:
' 1. Open a model document with a simple fillet feature.
' 2. Select the simple fillet feature.
' 3. Open the Immediate window.
' 4. Run the macro.
'
' Postconditions: Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swFillet As SimpleFilletFeatureData2

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swFillet = swFeat.GetDefinition

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print("Feature = " & swFeat.Name)
        Debug.Print("    Constant width? " & swFillet.ConstantWidth)
        Debug.Print("    Curvature continuous? " & swFillet.CurvatureContinuous)
        Debug.Print("    Default radius = " & swFillet.DefaultRadius)
        Debug.Print("    Number of fillet items = " & swFillet.FilletItemsCount)
        Debug.Print("    Multiple radii? " & swFillet.IsMultipleRadius)
        Debug.Print("    Keep existing features? " & swFillet.KeepFeatures)
        Debug.Print("    Apply fillet to attachment edges? " & swFillet.OmitAttachedEdges)
        Debug.Print("    Overflow type = " & swFillet.OverflowType)
        Debug.Print("    Extend fillet to all affected parts? " & swFillet.PropagateFeatureToParts)
        Debug.Print("    Extend fillet to all tangent faces? " & swFillet.PropagateToTangentFaces)
        Debug.Print("    Reverse normal? " & swFillet.ReverseFaceNormal(0))
        Debug.Print("    Round fillet corners? " & swFillet.RoundCorners)
        Debug.Print("    Trim and attach to surfaces? " & swFillet.TrimAndAttachSurfaces)
        Debug.Print("    Type of fillet? " & swFillet.Type)
        Debug.Print("    Number of faces associated with this fillet = " &
        swFillet.GetFaceCount(swSimpleFilletWhichFaces_e.swSimpleFilletSingleRadius)
    
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

## Get Data for Surface-extend Feature Example (C#)

This example shows how to get the data for the selected surface-extend feature.

```
//-----
// Preconditions:
// 1. Open a model document with a surface-extend feature.
// 2. Select the surface-extend feature.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace SurfaceExtendFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            SurfaceExtendFeatureData swSurfExtend = default(SurfaceExtendFeatureData);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSurfExtend = (SurfaceExtendFeatureData)swFeat.GetDefinition();

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("Feature = " + swFeat.Name);
            Debug.Print("    Extended distance = " + swSurfExtend.Distance);
            Debug.Print("    Type of surface-extend feature = " + swSurfExtend.Type);

        }

        public SldWorks swApp;
    }
}
```

### Get Data for Surface-extend Feature Example (VBA)

This example shows how to get the data for the selected surface-extend feature.

```
'-----  
' Preconditions:  
' 1. Open a model document with a surface-extend feature.  
' 2. Select the surface-extend feature.  
'  
' Postconditions: Inspect the Immediate window.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swSelMgr              As SldWorks.SelectionMgr  
    Dim swFeat               As SldWorks.Feature  
    Dim swSurfExtend          As SldWorks.SurfaceExtendFeatureData
```

```
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Set swSurfExtend = swFeat.GetDefinition
```

```
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "Feature = " & swFeat.Name  
    Debug.Print "    Extended distance = " & swSurfExtend.Distance  
    Debug.Print "    Type of surface-extend feature = " & swSurfExtend.Type
```

End Sub



### Get Data for Surface-extend Feature Example (VB.NET)

This example shows how to get the data for the selected surface-extend feature.

```
'-----  
' Preconditions:  
' 1. Open a model document with a surface-extend feature.  
' 2. Select the surface-extend feature.  
'-----  
' Postconditions: Inspect the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim swFeat As Feature  
        Dim swSurfExtend As SurfaceExtendFeatureData  
  
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
        swFeat = swSelMgr.GetSelectedObject6(1, -1)  
        swSurfExtend = swFeat.GetDefinition  
  
        Debug.Print("File = " & swModel.GetPathName)  
        Debug.Print("Feature = " & swFeat.Name)  
        Debug.Print("    Extended distance = " & swSurfExtend.Distance)  
        Debug.Print("    Type of surface-extend feature = " & swSurfExtend.Type)  
  
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

## Get Data for Surface-flatten Feature Example (C#)

This example shows how to insert a surface-flatten feature and get its data.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Inserts a surface-flatten feature.
// 3. Gets surface-flatten feature data.
// 4. Examine the graphics area and FeatureManager design
//    tree.
//
// NOTE: Because this part is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            SurfaceFlattenFeatureData swSurfaceFlattenFeatureData = default(SurfaceFlattenFeatureData);
            Edge swEdge = default(Edge);
            bool status = false;
            int warnings = 0;
            int errors = 0;
            string fileName = null;
            object[] edges = null;
            int nbrTearEdges = 0;
            int i = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\hemisphere.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Select face, vertex, and curves for surface-flatten feature
            status = swModelDocExt.SelectByID2("", "FACE", 0.00546207138008786, -0.0373996629648742, 0.0130049636270216, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", 0, -0.0400014251170795, -0.00054185036618795, true, 16, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.011047389592717, -6.23453597697887E-06, 0.0382739927527045, true, 8, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0108290857690036, -0.0262767618508641, 0.0280128973987478, true, 8, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.00907918330954303, 0.0286363967873063, 0.0262553195430037, true, 8, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0157595492108076, 0.0240950142317162, -0.0272880335909787, true, 8, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0143808110794609, -0.0281074423709366, -0.0240578702886196, true, 8, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.01886657492451, 2.48139106812095E-05, -0.035004838468241, true, 8, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0140143161426676, -0.0374399859522778, 0.000123652313243445, true, 8, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0164192327552536, 0.0363245554251959, -9.58561062339106E-05, true, 8, null, 0);

            // Create surface-flatten feature
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.InsertFlattenSurface2(8, true);

            // Get some surface-flatten feature data
            swSurfaceFlattenFeatureData = (SurfaceFlattenFeatureData)swFeature.GetDefinition();
            swSurfaceFlattenFeatureData.AccessSelections(swModel, null);
            Debug.Print("Feature = " + swFeature.Name);
            Debug.Print("Accuracy of flattened triangle mesh = " + swSurfaceFlattenFeatureData.AccuracyFactor);
            Debug.Print("Keep internal control curves? " + swSurfaceFlattenFeatureData.KeepInternalControlCurves);
            if (swSurfaceFlattenFeatureData.ShouldMakeTears == true)
            {
                Debug.Print("Tear edges: ");
                edges = (object[])swSurfaceFlattenFeatureData.TearEdges;
                nbrTearEdges = edges.Length;
                Debug.Print("Number of tear edges: " + nbrTearEdges);
                for (i = 0; i <= nbrTearEdges - 1; i++)
                {
                    swEdge = (Edge)edges[i];
                    SelectionMgr swSelMgr = default(SelectionMgr);
                    swSelMgr = (SelectionMgr)swModel.SelectionManager;
                    status = swSelMgr.AddSelectionListObject(swEdge, null);
                    Debug.Print("Type of object (51 = swSelREFEDGES): " + swSelMgr.GetSelectedObjectType3(1, -1));
                }
            }
            swSurfaceFlattenFeatureData.ReleaseSelectionAccess();
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

## Get Data for Surface-flatten Feature Example (VBA)

This example shows how to insert a surface-flatten feature and get its data.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Inserts a surface-flatten feature.
' 3. Gets surface-flatten feature data.
' 4. Examines the graphics area and FeatureManager design
'    tree.
'
' NOTE: Because this part is used elsewhere, do not save
'       changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swSurfaceFlattenFeatureData As SldWorks.SurfaceFlattenFeatureData
Dim swEdge As SldWorks.Edge
Dim status As Boolean
Dim warnings As Long
Dim errors As Long
Dim fileName As String
Dim edges As Variant
Dim nbrTearEdges As Long
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\Hemisphere.SLDPRT"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    ' Select face, point, and curves for surface-flatten feature
    status = swModelDocExt.SelectByID("", "FACE", 5.46207138008786E-03, -3.73996629648742E-02, 1.30049636270216E-02, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID("", "VERTEX", 0, -4.00014251170795E-02, -5.4185036618795E-04, True, 16, Nothing, 0)
    status = swModelDocExt.SelectByID("", "EDGE", 0.011047389592717, -6.23453597697887E-06, 3.82739927527045E-02, True, 8, Nothing, 0)
    status = swModelDocExt.SelectByID("", "EDGE", 1.08290857690036E-02, -2.62767618508641E-02, 2.80128973987478E-02, True, 8, Nothing, 0)
    status = swModelDocExt.SelectByID("", "EDGE", 9.07918330954303E-03, 2.86363967873063E-02, 2.62553195430037E-02, True, 8, Nothing, 0)
    status = swModelDocExt.SelectByID("", "EDGE", 1.57595492108076E-02, 2.40950142317162E-02, -2.72880335909787E-02, True, 8, Nothing, 0)
    status = swModelDocExt.SelectByID("", "EDGE", 1.43808110794609E-02, -2.81074423709366E-02, -2.40578702886196E-02, True, 8, Nothing, 0)
    status = swModelDocExt.SelectByID("", "EDGE", 0.01886657492451, 2.48139106812095E-05, -0.035004838468241, True, 8, Nothing, 0)
    status = swModelDocExt.SelectByID("", "EDGE", 1.40143161426676E-02, -3.74399859522778E-02, 1.23652313243445E-04, True, 8, Nothing, 0)
    status = swModelDocExt.SelectByID("", "EDGE", 1.64192327552536E-02, 3.63245554251959E-02, -9.58561062339106E-05, True, 8, Nothing, 0)

    ' Create surface-flatten feature
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.InsertFlattenSurface2(8, True)

    ' Get surface-flatten feature data
    Set swSurfaceFlattenFeatureData = swFeature.GetDefinition
    swSurfaceFlattenFeatureData.AccessSelections swModel, Nothing
    Debug.Print "Feature = " & swFeature.Name
    Debug.Print "    Accuracy of flattened triangle mesh = " & swSurfaceFlattenFeatureData.AccuracyFactor
    Debug.Print "    Keep internal control curves? " & swSurfaceFlattenFeatureData.KeepInternalControlCurves
    If swSurfaceFlattenFeatureData.ShouldMakeTears = True Then
        Debug.Print "    Tear edges: "
        edges = swSurfaceFlattenFeatureData.TearEdges
        nbrTearEdges = UBound(edges) + 1
        Debug.Print "    Number of tear edges: " & nbrTearEdges
        For i = 0 To UBound(edges)
            Set swEdge = edges(i)
            Dim swSelMgr As SldWorks.SelectionMgr
            Set swSelMgr = swModel.SelectionManager
            status = swSelMgr.AddSelectionListObject(swEdge, Nothing)
            Debug.Print "    Type of object (51 = swSelREFEDGES): " & swSelMgr.GetSelectedObjectType3(1, -1)
        Next i
    End If
    swSurfaceFlattenFeatureData.ReleaseSelectionAccess

End Sub

```

## Get Data for Surface-flatten Feature Example (VB.NET)

This example shows how to insert a surface-flatten feature and get its data.

```

-----
Preconditions:
1. Verify that the part to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the part.
2. Inserts a surface-flatten feature.
3. Gets surface-flatten feature data.
4. Examine the graphics area and FeatureManager design tree.
NOTE: Because this part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swSurfaceFlattenFeatureData As SurfaceFlattenFeatureData
        Dim swEdge As Edge
        Dim status As Boolean
        Dim warnings As Integer
        Dim errors As Integer
        Dim fileName As String
        Dim edges As Object
        Dim nbrTearEdges As Integer
        Dim i As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\Hemisphere.SLDPRPT"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        ' Select face, vertex, and curves for surface-flatten feature
        status = swModelDocExt.SelectByID("", "FACE", 0.00546207138008786, -0.0373996629648742, 0.0130049636270216, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID("", "VERTEX", 0, -0.0400014251170795, -0.00054185036618795, True, 16, Nothing, 0)
        status = swModelDocExt.SelectByID("", "EDGE", 0.011047389592717, -0.00000623453597697887, 0.0382739927527045, True, 8, Nothing, 0)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0108290857690036, -0.0262767618508641, 0.0280128973987478, True, 8, Nothing, 0)
        status = swModelDocExt.SelectByID("", "EDGE", 0.00907918330954303, 0.0286363967873063, 0.0262553195430037, True, 8, Nothing, 0)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0157595492108076, 0.0240950142317162, -0.0272880335909787, True, 8, Nothing, 0)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0143808110794609, -0.0281074423709366, -0.0240578702886196, True, 8, Nothing, 0)
        status = swModelDocExt.SelectByID("", "EDGE", 0.01886657492451, 0.0000248139106812095, -0.035004838468241, True, 8, Nothing, 0)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0140143161426676, -0.0374399859522778, 0.000123652313243445, True, 8, Nothing, 0)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0164192327552536, 0.0363245554251959, -0.0000958561062339106, True, 8, Nothing, 0)

        ' Create surface-flatten feature
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.InsertFlattenSurface2(8, True)

        ' Get some surface-flatten feature data
        swSurfaceFlattenFeatureData = swFeature.GetDefinition
        swSurfaceFlattenFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Feature = " & swFeature.Name)
        Debug.Print(" Accuracy of flattened triangle mesh = " & swSurfaceFlattenFeatureData.AccuracyFactor)
        Debug.Print(" Keep internal control curves? " & swSurfaceFlattenFeatureData.KeepInternalControlCurves)
        If swSurfaceFlattenFeatureData.ShouldMakeTears = True Then
            Debug.Print(" Tear edges: ")
            edges = swSurfaceFlattenFeatureData.TearEdges
            nbrTearEdges = UBound(edges) + 1
            Debug.Print(" Number of tear edges: " & nbrTearEdges)
            For i = 0 To UBound(edges)
                swEdge = edges(i)
                Dim swSelMgr As SelectionMgr
                swSelMgr = swModel.SelectionManager
                status = swSelMgr.AddSelectionListObject(swEdge, Nothing)
                Debug.Print(" Type of object (51 = swSeIREFEDGES): " & swSelMgr.GetSelectedObjectType3(1, -1))
            Next i
        End If
        swSurfaceFlattenFeatureData.ReleaseSelectionAccess()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

## Get Data for Surface Revolve Feature Example (C#)

This example shows how to get the data for the selected surface revolve feature.

```
//-----
// Preconditions:
// 1. Open a model document with a surface revolve feature.
// 2. Select the surface revolve feature.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace SurfaceRevolveFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            SurfRevolveFeatureData swSurfRevolve = default(SurfRevolveFeatureData);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSurfRevolve = (SurfRevolveFeatureData)swFeat.GetDefinition();

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("Feature = " + swFeat.Name);
            Debug.Print("    Reverse direction? " + swSurfRevolve.ReverseDirection);
            Debug.Print("    Type of surface revolve = " + swSurfRevolve.Type);

        }

        public SldWorks swApp;
    }
}
```

### Get Data for Surface Revolve Feature Example (VBA)

This example shows how to get the data for the selected surface revolve feature.

```
'-----  
' Preconditions:  
' 1. Open a model document with a surface revolve feature.  
' 2. Select the surface revolve feature.  
'  
' Postconditions: Inspect the Immediate window.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swSelMgr              As SldWorks.SelectionMgr  
    Dim swFeat               As SldWorks.Feature  
    Dim swSurfRevolve        As SldWorks.SurfRevolveFeatureData
```

```
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Set swSurfRevolve = swFeat.GetDefinition
```

```
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "Feature = " & swFeat.Name  
    Debug.Print "    Reverse direction? " & swSurfRevolve.ReverseDirection  
    Debug.Print "    Type of surface revolve = " & swSurfRevolve.Type
```

End Sub

### Get Data for Surface Revolve Feature Example (VB.NET)

This example shows how to get the data for the selected surface revolve feature.

```
'-----  
' Preconditions:  
' 1. Open a model document with a surface revolve feature.  
' 2. Select the surface revolve feature.  
'-----
```

```
' Postconditions: Inspect the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swSelMgr As SelectionMgr
```

```
        Dim swFeat As Feature
```

```
        Dim swSurfRevolve As SurfRevolveFeatureData
```

```
        swModel = swApp.ActiveDoc
```

```
        swSelMgr = swModel.SelectionManager
```

```
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
```

```
        swSurfRevolve = swFeat.GetDefinition
```

```
        Debug.Print("File = " & swModel.GetPathName)
```

```
        Debug.Print("Feature = " & swFeat.Name)
```

```
        Debug.Print("    Reverse direction? " & swSurfRevolve.ReverseDirection)
```

```
        Debug.Print("    Type of surface revolve = " & swSurfRevolve.Type)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

## Get Data for Wrap Feature Example (C#)

This example shows how to get data for a wrap feature.

```
//-----
// Preconditions:
// 1. Open a model document with a wrap feature.
// 2. Select the wrap feature.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace WrapFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            WrapSketchFeatureData swWrapSketch = default(WrapSketchFeatureData);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swWrapSketch = (WrapSketchFeatureData)swFeat.GetDefinition();

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("Feature = " + swFeat.Name);
            Debug.Print("    Reverse direction? " + swWrapSketch.ReverseDirection);
            Debug.Print("    Type of surface revolve = " + swWrapSketch.Type);
            Debug.Print("    Thickness = " + swWrapSketch.Thickness);

        }

        public SldWorks swApp;
    }
}
```



### Get Data for Wrap Feature Example (VBA)

This example shows how to get data for a wrap feature.

```
'-----  
' Preconditions:  
' 1. Open a model document with a wrap feature.  
' 2. Select the wrap feature.  
'-----  
' Postconditions: Inspect the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swSelMgr              As SldWorks.SelectionMgr  
    Dim swFeat               As SldWorks.Feature  
    Dim swWrapSketch         As SldWorks.WrapSketchFeatureData  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Set swWrapSketch = swFeat.GetDefinition  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "Feature = " & swFeat.Name  
    Debug.Print "    Reverse direction? " & swWrapSketch.ReverseDirection  
    Debug.Print "    Type of surface revolve = " & swWrapSketch.Type  
    Debug.Print "    Thickness = " & swWrapSketch.Thickness  
  
End Sub
```

### Get Data for Wrap Feature Example (VB.NET)

This example shows how to get data for a wrap feature.

```
'-----  
' Preconditions:  
' 1. Open a model document with a wrap feature.  
' 2. Select the wrap feature.  
'-----  
' Postconditions: Inspect the Immediate window.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim swFeat As Feature  
        Dim swWrapSketch As WrapSketchFeatureData  
  
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
        swFeat = swSelMgr.GetSelectedObject6(1, -1)  
        swWrapSketch = swFeat.GetDefinition  
  
        Debug.Print("File = " & swModel.GetPathName)  
        Debug.Print("Feature = " & swFeat.Name)  
        Debug.Print("    Reverse direction? " & swWrapSketch.ReverseDirection)  
        Debug.Print("    Type of surface revolve = " & swWrapSketch.Type)  
        Debug.Print("    Thickness = " & swWrapSketch.Thickness)  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```

### Get Dependencies for Open and Unopened Documents Example (VBA)

This example shows how to get the names of any dependencies for an open or unopened document.

```
'-----
' Preconditions:
' 1. Verify that the specified drawing document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the dependencies for the specified drawing
'    document.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()
    ' Name of unopened document
    Const sDefaultName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim sDocName As String
    Dim vDepend As Variant
    Dim bRet As Boolean
    Dim i As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    If Not swModel Is Nothing Then
        sDocName = swModel.GetPathName
    Else
        sDocName = sDefaultName
    End If

    vDepend = swApp.GetDocumentDependencies2(sDocName, True, True, False)
    Debug.Print sDocName
    If IsEmpty(vDepend) Then
        Debug.Print "    No dependencies."
        Exit Sub
    End If

    For i = 0 To (UBound(vDepend) - 1) / 2
        Debug.Print "        " + vDepend(2 * i) + " --> " + vDepend(2 * i + 1)
    Next i
End Sub
```

## Get Depth of Extrusion Example (C#)

This example shows how to get the depth of an extrusion.

```
//-----
// Preconditions:
// 1. Open a part containing a Boss-Extrude1 feature.
// 2. Open the Immediate window.
//
// Postconditions: Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace DimGetSystemValue3_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Dimension swDim = default(Dimension);
            object vConfigNames = null;
            double[] vValue = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDim = (Dimension)swModel.Parameter("D1@Boss-Extrude1");

            Debug.Assert((swDim != null));

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("  Full name = " + swDim.FullName);
            Debug.Print("   Name = " + swDim.Name);

            vConfigNames = swModel.GetConfigurationNames();
            vValue = (double[])swDim.GetSystemValue3((int)swInConfigurationOpts_e.swThisConfiguration, (vConfigNames));

            Debug.Print("  System value = " + vValue[0] * 1000.0 + " " + " mm");
        }
    }
    public SldWorks swApp;
}
```

### Get Depth of Extrusion Example (VBA)

This example shows how to get the depth of an extrusion.

```
'-----  
' Preconditions:  
' 1. Open a part containing a Boss-Extrude1 feature.  
' 2. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDim As SldWorks.Dimension  
    Dim vConfigNames As Variant  
    Dim vValue As Variant  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDim = swModel.Parameter("D1@Boss-Extrude1")  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " Full name = " & swDim.FullName  
    Debug.Print " Name = " & swDim.Name  
  
    vConfigNames = swModel.GetConfigurationNames  
    vValue = swDim.GetSystemValue3(swThisConfiguration, (vConfigNames))  
  
    Debug.Print " System value = " & vValue(0) * 1000#; "" & " mm"  
  
End Sub
```

### Get Depth of Extrusion Example (VB.NET)

This example shows how to get the depth of an extrusion.

```
'-----  
' Preconditions:  
' 1. Open a part containing a Boss-Extrude1 feature.  
' 2. Open the Immediate window.  
'-----
```

```
' Postconditions: Examine the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2  
        Dim swDim As Dimension  
        Dim vConfigNames As Object  
        Dim vValue As Object
```

```
        swModel = swApp.ActiveDoc  
        swDim = swModel.Parameter("D1@Boss-Extrude1")
```

```
        Debug.Assert(Not swDim Is Nothing)
```

```
        Debug.Print("File = " & swModel.GetPathName)  
        Debug.Print("  Full name = " & swDim.FullName)  
        Debug.Print("  Name = " & swDim.Name)
```

```
        vConfigNames = swModel.GetConfigurationNames  
        vValue = swDim.GetSystemValue3(swInConfigurationOpts_e.swThisConfiguration, (vConfigNames))
```

```
        Debug.Print("  System value = " & vValue(0) * 1000.0# & " " & " mm")
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

## Get Design Table Example (C#)

This example shows how to get a design table and its contents.

```
//-----
// Preconditions:
// 1. Open a part or assembly document that
//    contains a design table.
// 2. Verify that a design table exists by
//    expanding Tables in the ConfigurationManager.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Prints the design table contents to the
//    Immediate window.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetTitleDesignTableCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DesignTable swDesTable = default(DesignTable);
            int nTotRow = 0;
            int nTotCol = 0;
            string sRowStr = null;
            int i = 0;
            int j = 0;
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            swDesTable = (DesignTable)swModel.GetDesignTable();
            bRet = swDesTable.Attach();

            nTotRow = swDesTable.GetTotalRowCount();
            nTotCol = swDesTable.GetTotalColumnCount();
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" Title      = " + swDesTable.GetTitle());
            Debug.Print(" Row       = " + swDesTable.GetRowCount());
            Debug.Print(" Col      = " + swDesTable.GetColumnCount());
            Debug.Print(" TotRow   = " + nTotRow);
            Debug.Print(" TotCol   = " + nTotCol);
            Debug.Print(" VisRow   = " + swDesTable.GetVisibleRowCount());
            Debug.Print(" VisCol   = " + swDesTable.GetVisibleColumnCount());
            Debug.Print("");

            for (i = 0; i <= nTotRow; i++)
            {
                sRowStr = " |";
                for (j = 0; j <= nTotCol; j++)
                {
                    sRowStr = sRowStr + swDesTable.GetEntryText(i, j) + "|";
                }
                Debug.Print(sRowStr);
            }
            swDesTable.Detach();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

## Get Design Table Example (VBA)

This example shows how to get a design table and its contents.

```
'-----
' Preconditions:
' 1. Open a part or assembly document that
'    contains a design table.
' 2. Verify that a design table exists by
'    expanding Tables in the ConfigurationManager.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Prints the design table contents to the
'    Immediate window.
' 2. Examine the Immediate window.
'-----
```

Option Explicit

Sub main()

```
Dim swApp                As SldWorks.SldWorks
Dim swModel              As SldWorks.ModelDoc2
Dim swDesTable           As SldWorks.DesignTable
Dim nTotRow              As Long
Dim nTotCol              As Long
Dim sRowStr              As String
Dim i                   As Long
Dim j                   As Long
Dim bRet                 As Boolean

Set swApp = CreateObject("SldWorks.Application")
Set swModel = swApp.ActiveDoc

Set swDesTable = swModel.GetDesignTable
bRet = swDesTable.Attach

Debug.Assert bRet

nTotRow = swDesTable.GetTotalRowCount
nTotCol = swDesTable.GetTotalColumnCount

Debug.Print "File = " & swModel.GetPathName
Debug.Print "  Title      = " & swDesTable.GetTitle
Debug.Print "  Row       = " & swDesTable.GetRowCount
Debug.Print "  Col       = " & swDesTable.GetColumnCount
Debug.Print "  TotRow    = " & nTotRow
Debug.Print "  TotCol    = " & nTotCol
Debug.Print "  VisRow    = " & swDesTable.GetVisibleRowCount
Debug.Print "  VisCol    = " & swDesTable.GetVisibleColumnCount
Debug.Print ""
```

```
For i = 0 To nTotRow
    sRowStr = " |"

    For j = 0 To nTotCol
        sRowStr = sRowStr + swDesTable.GetEntryText(i, j) + "|"
    Next j

    Debug.Print sRowStr

Next i

swDesTable.Detach
```

End Sub



## Get Design Table Example (VB.NET)

This example shows how to get a design table and its contents.

```

-----
Preconditions:
1. Open a part or assembly document that
   contains a design table.
2. Verify that a design table exists by
   expanding Tables in the ConfigurationManager.
3. Open the Immediate window.
-----
Postconditions:
1. Prints the design table contents to the
   Immediate window.
2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swDesTable As DesignTable
        Dim nTotRow As Integer
        Dim nTotCol As Integer
        Dim sRowStr As String
        Dim i As Integer
        Dim j As Integer
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc

        swDesTable = swModel.GetDesignTable
        bRet = swDesTable.Attach

        nTotRow = swDesTable.GetTotalRowCount
        nTotCol = swDesTable.GetTotalColumnCount
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" Title      = " & swDesTable.GetTitle)
        Debug.Print(" Row       = " & swDesTable.GetRowCount)
        Debug.Print(" Col       = " & swDesTable.GetColumnCount)
        Debug.Print(" TotRow    = " & nTotRow)
        Debug.Print(" TotCol    = " & nTotCol)
        Debug.Print(" VisRow    = " & swDesTable.GetVisibleRowCount)
        Debug.Print(" VisCol    = " & swDesTable.GetVisibleColumnCount)
        Debug.Print("")

        For i = 0 To nTotRow
            sRowStr = " |"
            For j = 0 To nTotCol
                sRowStr = sRowStr + swDesTable.GetEntryText(i, j) + "|"
            Next j
            Debug.Print(sRowStr)
        Next i
        swDesTable.Detach()
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

## Get Detail Circle Information Example (VBA)

This example shows how to get information about detail circles.

```
'-----
'
' Preconditions:
'   (1) Drawing document is open.
'   (2) Drawing view to which you added at
'       least one detail circle is selected.
'
' Postconditions: None
'
'-----

Option Explicit
Sub main()
    Dim swApp           As SldWorks.SldWorks
    Dim swModel          As SldWorks.ModelDoc2
    Dim swDraw           As SldWorks.DrawingDoc
    Dim swAssy           As SldWorks.AssemblyDoc
    Dim swSelMgr         As SldWorks.SelectionMgr
    Dim swView           As SldWorks.View
    Dim vDetCirc         As Variant
    Dim nSize            As Long
    Dim nNumDetCirc      As Long
    Dim i                As Long
    Dim j                As Long
    Dim index            As Long
    Dim nNumArrow        As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject5(1)

    "View::GetDetailCircleInfo2
    ' [ numDetailCircles,
    '   [ layer, centerPt[3], startPt[3], endPt[3], lineType, textPt[3], textHeight,
    '     numArrows,
    '     [ arrowTip[3], arrowComponent[3], arrowWidth, arrowHeight, arrowStyle ]
    ' ]
    ' ]

    vDetCirc = swView.GetDetailCircleInfo2
    nNumDetCirc = swView.GetDetailCircleCount2(nSize)
    Debug.Assert nNumDetCirc = vDetCirc(0)
    Debug.Assert nSize = UBound(vDetCirc) + 1
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " View = " & swView.Name

    index = 1 ' Set index to start of first detail circle
    For i = 0 To nNumDetCirc - 1
        Debug.Print " Circle(" & i & ")"
        Debug.Print " Layer      = " & vDetCirc(index + 0)
        Debug.Print " Center pt  = (" & vDetCirc(index + 1) * 1000# & ", " & vDetCirc(index + 2) * 1000# & ", " & vDetCirc(index + 3) * 1000# & ") mm"
        Debug.Print " Start pt   = (" & vDetCirc(index + 4) * 1000# & ", " & vDetCirc(index + 5) * 1000# & ", " & vDetCirc(index + 6) * 1000# & ") mm"
        Debug.Print " End pt     = (" & vDetCirc(index + 7) * 1000# & ", " & vDetCirc(index + 8) * 1000# & ", " & vDetCirc(index + 9) * 1000# & ") mm"
        Debug.Print " Line type  = " & vDetCirc(index + 10)
        Debug.Print " Text pt    = (" & vDetCirc(index + 11) * 1000# & ", " & vDetCirc(index + 12) * 1000# & ", " & vDetCirc(index + 13) * 1000# & ") mm"
        Debug.Print " Text ht    = " & vDetCirc(index + 14) * 1000# & " mm"

        nNumArrow = vDetCirc(index + 15)
        index = index + 16 ' Set index to start of first arrow
        For j = 0 To nNumArrow - 1
            Debug.Print " Arrow(" & j & ")"
            Debug.Print " Tip pt    = (" & vDetCirc(index + 0) * 1000# & ", " & vDetCirc(index + 1) * 1000# & ", " & vDetCirc(index + 2) * 1000# & ") mm"
            Debug.Print " Comp pt   = (" & vDetCirc(index + 3) * 1000# & ", " & vDetCirc(index + 4) * 1000# & ", " & vDetCirc(index + 5) * 1000# & ") mm"
            Debug.Print " Width     = " & vDetCirc(index + 6) * 1000# & " mm"
```

```
Debug.Print "    Height    =" & vDetCirc(index + 7) * 1000# & " mm"  
Debug.Print "    Style     =" & vDetCirc(index + 8)
```

```
index = index + 9 ' Set index to start of next arrow  
Next j
```

```
' Index should now be at start of next detail circle  
Next i  
End Sub  
'.-----
```

## Get Differences Between Parts Example (VBA)

This examples shows how to get the differences between two parts.

**NOTE:** When a part is designed for injection molding, many small design details must be added for the injection molding process. Typically, these are draft angles, thickening of walls, additional ribs, and so on. Quite often, without specialized knowledge and experience, these are added to the model, by mold maker, after the part has been designed by a designer. It is often desirable to find out what changes have been made to the original model. This is usually done to determine the change in the amount of material for the revised part. This example uses several geometry- and topology-related methods to find the differences and reverse difference between two parts.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\box.sldprt and holcube.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates two new parts showing the differences between the parts.
' 2. Examine the Immediate window.
' 3. Examine the graphics area, then click Window and open the other
'    just-created part.
'
' NOTES:
' * This example is not intended to be a replacement for SOLIDWORKS
'   Utilities comparison utility.
' * New parts can contain one or more solid bodies.
' * This code is intended to run on parts that are similar.
' * Because the parts are used elsewhere, do not save changes.
'-----
Option Explicit

Sub CreateDiffPart(swApp As SldWorks.SldWorks, vBodyResArr As Variant)
    Dim vBodyRes As Variant
    Dim swBodyRes As SldWorks.Body2
    Dim swPartRes As SldWorks.PartDoc
    Dim swFeatRes As SldWorks.Feature

    If IsEmpty(vBodyResArr) Then Exit Sub

    Set swPartRes = swApp.NewPart
    For Each vBodyRes In vBodyResArr
        Set swBodyRes = vBodyRes
        Set swFeatRes = swPartRes.CreateFeatureFromBody3(swBodyRes, False, swCreateFeatureBodyCheck + swCreateFeatureBodySimplify): Debug.Assert Not
swFeatRes Is Nothing
    Next

End Sub

Sub main()

    Const sPartName1 As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
    Const sPartName2 As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\holcube.sldprt"
    Dim swApp As SldWorks.SldWorks
    Dim swModel1 As SldWorks.ModelDoc2
    Dim swPart1 As SldWorks.PartDoc
    Dim swBody1 As SldWorks.Body2
    Dim swBody1Copy As SldWorks.Body2
    Dim swBody1Copy2 As SldWorks.Body2
    Dim swModel2 As SldWorks.ModelDoc2
    Dim swPart2 As SldWorks.PartDoc
    Dim swBody2 As SldWorks.Body2
    Dim swBody2Copy As SldWorks.Body2
    Dim swBody2Copy2 As SldWorks.Body2
    Dim vswBody1 As Variant
    Dim vswBody2 As Variant
    Dim vBodyResArr1 As Variant
    Dim vBodyResArr2 As Variant
    Dim nRetVal As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel1 = swApp.GetOpenDocumentByName(sPartName1)
    Set swModel2 = swApp.GetOpenDocumentByName(sPartName2)
    Set swPart1 = swModel1
    Set swPart2 = swModel2

    ' Get all of the visible bodies in each part
    vswBody1 = swPart1.GetBodies2(swSolidBody, True)
    vswBody2 = swPart2.GetBodies2(swSolidBody, True)
    Set swBody1 = vswBody1(0)
    Set swBody2 = vswBody2(0)
    Set swBody1Copy = swBody1.Copy
    Set swBody2Copy = swBody2.Copy
    Set swBody1Copy2 = swBody1.Copy
    Set swBody2Copy2 = swBody2.Copy

    vBodyResArr1 = swBody1Copy2.Operations2(SwBODYCUT, swBody2Copy2, nRetVal): Debug.Assert swBodyOperationNoError = nRetVal
    vBodyResArr2 = swBody2Copy.Operations2(SwBODYCUT, swBody1Copy, nRetVal): Debug.Assert swBodyOperationNoError = nRetVal

    If IsEmpty(vBodyResArr1) And IsEmpty(vBodyResArr2) Then Exit Sub

    Debug.Print "Part1          = " & swModel1.GetPathName
    Debug.Print "Part2          = " & swModel2.GetPathName

    If Not IsEmpty(vBodyResArr1) Then
        Debug.Print "  Diffs(1-2)   = " & UBound(vBodyResArr1) + 1
    
```

```
End If

If Not IsEmpty(vBodyResArr2) Then
    Debug.Print " Diffs(2-1)  = " & UBound(vBodyResArr2) + 1
End If

CreateDiffPart swApp, vBodyResArr1
CreateDiffPart swApp, vBodyResArr2

End Sub
```

### Get Dimension of Distance Mate Example (VBA)

This example shows how to get the dimension value of a distance mate in an assembly.

**NOTE:** You can use this sample code to get all the dimension values for any selected feature in the FeatureManager design tree.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\motor casing and switch top plate.sldasm.  
' 2. Expand MateGroup1 and select Distance8.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the dimension value of Distance8.  
' 2. Examine the Immediate window.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Sub ProcessFeature(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swFeat As SldWorks.Feature)  
    Dim swDispDim As SldWorks.DisplayDimension  
    Dim swDim As SldWorks.Dimension  
    Dim dimValue As Variant  
  
    Debug.Print " " + swFeat.Name  
  
    Set swDispDim = swFeat.GetFirstDisplayDimension  
    While (Not swDispDim Is Nothing)  
        Set swDim = swDispDim.GetDimension  
        dimValue = swDim.GetSystemValue3(swThisConfiguration, Empty)  
        Debug.Print " " + swDim.FullName + " = " & (dimValue(0) * 1000) & " mm "  
        Set swDispDim = swFeat.GetNextDisplayDimension(swDispDim)  
    Wend  
End Sub  
  
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeat As SldWorks.Feature  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Debug.Print "File = " & swModel.GetPathName  
    ProcessFeature swApp, swModel, swFeat  
End Sub
```

## Get Dimension Tolerance Example (C#)

This example shows how to get and set dimension tolerance values and options.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\box.slddrw.
// 2. Open the Immediate window.
// 3. Zoom in on the diameter dimension and click it.
//
// Postconditions:
// 1. Sets the dimension tolerance type to fit with tolerance.
// 2. Sets the dimension fit tolerance type to transitional.
// 3. Gets the minimum and maximum dimension tolerances for the
//    selected dimension and whether the values are valid.
// 4. Gets the height and scale of the font of the dimension
//    tolerances.
// 5. Sets the dimension fit tolerance font to be the same
//    size as the dimension tolerance font.
// 6. Gets the height and scale of the font of the dimension fit
//    tolerance.
// 7. Examine the Immediate window and drawing sheet.
//
// NOTE: Because this drawing is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            Dimension swDimension = default(Dimension);
            DimensionTolerance swDimensionTolerance = default(DimensionTolerance);
            double maxValue = 0;
            double minValue = 0;
            int maxValueValid = 0;
            int minValueValid = 0;
            double fontHeight = 0;
            double fontScale = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            //Get the selection
            swDisplayDimension = (DisplayDimension)swSelMgr.GetSelectedObject6(1, 0);

            //If selection is not a dimension, then exit
            if (swSelMgr.GetSelectedObjectType3(1, -1) != (int)swSelectType_e.swSelDIMENSIONS)
                return;

            //Selection is a dimension, so get the dimension tolerance object
            swDimension = (Dimension)swDisplayDimension.GetDimension2(0);
            swDimensionTolerance = (DimensionTolerance)swDimension.Tolerance;

            //Set tolerance type to fit with tolerance and transitional
            swDimensionTolerance.Type = (int)swTolType_e.swTolFITWITHTOL;
            Debug.Print("Type of tolerance (8 = swTolType_e.swTolFITWITHTOL): " + swDimensionTolerance.Type);
            swDimensionTolerance.FitType = (int)swFitType_e.swFitTRANSITIONAL;
            Debug.Print("Type of fit tolerance (2 = swFitType_e.swFitTransitional): " + swDimensionTolerance.FitType);
            Debug.Print("");

            //Get dimension tolerance minimum and maximum values
            //and whether values are valid
            maxValueValid = swDimensionTolerance.GetMaxValue2(out maxValue);
            minValueValid = swDimensionTolerance.GetMinValue2(out minValue);
            Debug.Print("Minimum dimension tolerance of " + minValue + " valid (0 = valid)? " + minValueValid);
            Debug.Print("Maximum dimension tolerance of " + maxValue + " valid (0 = valid)? " + maxValueValid);

            //Get some dimension tolerance values
            fontHeight = swDimensionTolerance.GetFontHeight();
            Debug.Print(" Height of font: " + fontHeight * 1000.0 + " mm");
            fontScale = swDimensionTolerance.GetFontScale();
            Debug.Print(" Scale of font: " + fontScale);
            Debug.Print("");

            //Set and get some dimension fit tolerance values
            swDimensionTolerance.SetFitFont(true, false, 0);
            Debug.Print("Dimension fit tolerance font same size as tolerance font? " + swDimensionTolerance.GetFitFontUseDimension());
            fontHeight = swDimensionTolerance.GetFitFontHeight();
            Debug.Print(" Height of fit tolerance font: " + fontHeight * 1000.0 + " mm");
            fontScale = swDimensionTolerance.GetFitFontScale();
            Debug.Print(" Scale of fit tolerance font: " + fontScale);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

## Get Dimension Tolerance Example (VBA)

This example shows how to get and set dimension tolerance values and options.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\box.slddrw.
' 2. Open the Immediate window.
' 3. Zoom in on the diameter dimension and click it.
'
' Postconditions:
' 1. Sets the dimension tolerance type to fit with tolerance.
' 2. Sets the dimension fit tolerance type to transitional.
' 3. Gets the minimum and maximum dimension tolerances for the
'    selected dimension and whether the values are valid.
' 4. Gets the height and scale of the font of the dimension
'    tolerances.
' 5. Sets whether the dimension fit tolerance font to be the same
'    size as the dimension tolerance font.
' 6. Gets the height and scale of the font of the dimension fit
'    tolerance.
' 7. Examine the Immediate window and drawing sheet.
'
' NOTE: Because this drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swDisplayDimension As SldWorks.DisplayDimension
Dim swDimension As SldWorks.Dimension
Dim swDimensionTolerance As SldWorks.DimensionTolerance
Dim maxValue As Double
Dim minValue As Double
Dim maxValueValid As Long
Dim minValueValid As Long
Dim fontHeight As Double
Dim fontScale As Double

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    'Get the selection
    Set swDisplayDimension = swSelMgr.GetSelectedObject6(1, 0)

    'If selection is not a dimension, then exit
    If swSelMgr.GetSelectedObjectType3(1, -1) <> swSelectType_e.swSelDIMENSIONS Then Exit Sub

    'Selection is a dimension, so get the dimension tolerance object
    Set swDimension = swDisplayDimension.GetDimension2(0)
    Set swDimensionTolerance = swDimension.Tolerance

    'Set tolerance type to fit with tolerance
    swDimensionTolerance.Type = swTolType_e.swTolFITWITHTOL
    Debug.Print "Type of tolerance (8 = swTolType_e.swTolFITWITHTOL): " & swDimensionTolerance.Type
    swDimensionTolerance.FitType = swFitType_e.swFitTRANSITIONAL
    Debug.Print "Type of fit tolerance (2 = swFitType_e.swFitTransitional): " & swDimensionTolerance.FitType
    Debug.Print ""

    'Get dimension tolerance minimum and maximum values
    'and whether values are valid
    maxValueValid = swDimensionTolerance.GetMaxValue2(maxValue)
    minValueValid = swDimensionTolerance.GetMinValue2(minValue)
    Debug.Print ("Minimum dimension tolerance of " & minValue & " valid (0 = valid)? " & minValueValid)
    Debug.Print ("Maximum dimension tolerance of " & maxValue & " valid (0 = valid)? " & maxValueValid)

    'Get some dimension tolerance values
    fontHeight = swDimensionTolerance.GetFontHeight
    Debug.Print (" Height of font: " & fontHeight * 1000# & " mm")
    fontScale = swDimensionTolerance.GetFontScale
    Debug.Print (" Scale of font: " & fontScale)
    Debug.Print ""

    'Set and get some dimension fit tolerance values
    swDimensionTolerance.SetFitFont True, False, 0
    Debug.Print ("Dimension fit tolerance font same size as tolerance font? " & swDimensionTolerance.GetFitFontUseDimension)
    fontHeight = swDimensionTolerance.GetFitFontHeight
    Debug.Print (" Height of fit tolerance font: " & fontHeight * 1000# & " mm")
    fontScale = swDimensionTolerance.GetFitFontScale
    Debug.Print (" Scale of fit tolerance font: " & fontScale)

End Sub

```



## Get Dimension Tolerance Example (VB.NET)

This example shows how to get and set dimension tolerance values and options.

```

Preconditions:
1. Open public_documents\samples\tutorial\api\box.slddrw.
2. Open the Immediate window.
3. Zoom in on the diameter dimension and click it.

Postconditions:
1. Sets the dimension tolerance type to fit with tolerance.
2. Sets the dimension fit tolerance type to transitional.
3. Gets the minimum and maximum dimension tolerances for the
   selected dimension and whether the values are valid.
4. Gets the height and scale of the font of the dimension
   tolerances.
5. Sets the dimension fit tolerance font to be the same
   size as the dimension tolerance font.
6. Gets the height and scale of the font of the dimension fit
   tolerance.
7. Examine the Immediate window and drawing sheet.

NOTE: Because this drawing is used elsewhere, do not save changes.

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swDisplayDimension As DisplayDimension
        Dim swDimension As Dimension
        Dim swDimensionTolerance As DimensionTolerance
        Dim maxValue As Double
        Dim minValue As Double
        Dim maxValueValid As Integer
        Dim minValueValid As Integer
        Dim fontHeight As Double
        Dim fontScale As Double

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager

        'Get the selection
        swDisplayDimension = swSelMgr.GetSelectedObject6(1, 0)

        'If selection is not a dimension, then exit
        If swSelMgr.GetSelectedObjectType3(1, -1) <> swSelectType_e.swSelDIMENSIONS Then Exit Sub

        'Selection is a dimension, so get the dimension tolerance object
        swDimension = swDisplayDimension.GetDimension2(0)
        swDimensionTolerance = swDimension.Tolerance

        'Set tolerance type to fit with tolerance and transitional
        swDimensionTolerance.Type = swTolType_e.swTolFITWITHTOL
        Debug.Print("Type of tolerance (8 = swTolType_e.swTolFITWITHTOL): " & swDimensionTolerance.Type)
        swDimensionTolerance.FitType = swFitType_e.swFitTRANSITIONAL
        Debug.Print("Type of fit tolerance (2 = swFitType_e.swFitTransitional): " & swDimensionTolerance.FitType)
        Debug.Print("")

        'Get dimension tolerance minimum and maximum values
        'and whether values are valid
        maxValueValid = swDimensionTolerance.GetMaxValue2(maxValue)
        minValueValid = swDimensionTolerance.GetMinValue2(minValue)
        Debug.Print("Minimum dimension tolerance of " & minValue & " valid (0 = valid)? " & minValueValid)
        Debug.Print("Maximum dimension tolerance of " & maxValue & " valid (0 = valid)? " & maxValueValid)

        'Get some dimension tolerance values
        fontHeight = swDimensionTolerance.GetFontHeight
        Debug.Print(" Height of font: " & fontHeight * 1000.0# & " mm")
        fontScale = swDimensionTolerance.GetFontScale
        Debug.Print(" Scale of font: " & fontScale)
        Debug.Print("")

        'Set and get some dimension fit tolerance values
        swDimensionTolerance.SetFitFont(True, False, 0)
        Debug.Print("Dimension fit tolerance font same size as tolerance font? " & swDimensionTolerance.GetFitFontUseDimension)
        fontHeight = swDimensionTolerance.GetFitFontHeight
        Debug.Print(" Height of fit tolerance font: " & fontHeight * 1000.0# & " mm")
        fontScale = swDimensionTolerance.GetFitFontScale
        Debug.Print(" Scale of fit tolerance font: " & fontScale)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

```
End Class
```

## Get Dimension Values Example (VBA)

This example shows how to get the dimension values in all configurations of a model.

```
'-----
'
' Preconditions:
'     (1) Model document with dimensions is open.
'     (2) Dimension is selected.
'
' Postconditions: None
'
'-----

Option Explicit

Public Enum swDimensionDrivenState_e
    swDimensionDrivenUnknown = 0 ' the driven/driving state is unknown
    swDimensionDriven = 1 ' the dimension is a driven dimension
    swDimensionDriving = 2 ' the dimension is a driving dimension
End Enum

Public Enum swInConfigurationOpts_e
    swConfigPropertySuppressFeatures = 0
    swThisConfiguration = 1
    swAllConfiguration = 2
    swSpecifyConfiguration = 3
End Enum

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swConfig As SldWorks.Configuration
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swDispDim As SldWorks.DisplayDimension
    Dim swDim As SldWorks.Dimension
    Dim vConfigNameArr As Variant
    Dim vConfigName As Variant
    Dim vDimValArr As Variant
    Dim vDimVal As Variant
    Dim sSpecConfigNameArr(0) As String
    Dim vSpecConfigNameArr As Variant
    Dim dimValue As Variant
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    Set swSelMgr = swModel.SelectionManager
    Set swDispDim = swSelMgr.GetSelectedObject5(1)
    Set swDim = swDispDim.GetDimension
    vConfigNameArr = swModel.GetConfigurationNames

    Debug.Print "File = " & swModel.GetPathName & "<" & swConfig.Name & ">"
    Debug.Print " " & swDim.FullName & " [" & swDim.Name & "]"
    dimValue = swDim.GetSystemValue3(swThisConfiguration, vConfigNameArr(0))
    Debug.Print " SystemValue = " & dimValue(0)
    Debug.Print " IsAppliedToAllConfigurations = " & swDim.IsAppliedToAllConfigurations
    Debug.Print " DrivenState = " & swDim.DrivenState
    Debug.Print " IsReference = " & swDim.IsReference
    Debug.Print " ReadOnly = " & swDim.ReadOnly

    For Each vConfigName In vConfigNameArr
        sSpecConfigNameArr(0) = vConfigName
        vSpecConfigNameArr = sSpecConfigNameArr
        vDimValArr = swDim.GetValue3(swSpecifyConfiguration, vSpecConfigNameArr): Debug.Assert 0 = UBound(vDimValArr)
        For Each vDimVal In vDimValArr
            Debug.Print " " & vConfigName & " --> " & vDimVal
        Next vDimVal
    Next vConfigName
End Sub
```

Next vConfigName

End Sub

.....

## Get Dimension Values in All Configurations Example (VBA)

This example shows how to get the dimension values in all configurations of a model.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\multiconfig-part-2.sldprt.
' 2. Right-click Annotations and click Show Feature Dimensions.
' 3. Open the Immediate window.
'
' Postconditions: Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swConfig As SldWorks.Configuration
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swDispDim As SldWorks.DisplayDimension
    Dim swDim As SldWorks.Dimension
    Dim vConfigNameArr As Variant
    Dim vConfigName As Variant
    Dim vDimValArr As Variant
    Dim vDimVal As Variant
    Dim sSpecConfigNameArr(0) As String
    Dim vSpecConfigNameArr As Variant
    Dim dimValue As Variant
    Dim status As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    Set swSelMgr = swModel.SelectionManager
    status = swModel.Extension.SelectByID2("D1@Extrude1@multiconfig-part-2.SLDPRT", "DIMENSION", 2.16557439463763E-02, -1.85256458254496E-02, 1.15860942898448E-03, False, 0, Nothing, 0)
    Set swDispDim = swSelMgr.GetSelectedObject6(1, -1)
    Set swDim = swDispDim.GetDimension
    vConfigNameArr = swModel.GetConfigurationNames

    Debug.Print "File = " & swModel.GetPathName & "<" & swConfig.Name & ">"
    Debug.Print " " & swDim.FullName & " [" & swDim.Name & "]"
    dimValue = swDim.GetSystemValue3(swThisConfiguration, vConfigNameArr(0))
    Debug.Print "    System value = " & dimValue(0)
    Debug.Print "    Is applied to all configurations? " & swDim.IsAppliedToAllConfigurations
    Debug.Print "    Driven state = " & swDim.DrivenState
    Debug.Print "    Is reference? " & swDim.IsReference
    Debug.Print "    Read only? " & swDim.ReadOnly

    Debug.Print "    Configuration names"
    For Each vConfigName In vConfigNameArr
        sSpecConfigNameArr(0) = vConfigName
        vSpecConfigNameArr = sSpecConfigNameArr
        vDimValArr = swDim.GetValue3(swSpecifyConfiguration, vSpecConfigNameArr): Debug.Assert 0 = UBound(vDimValArr)
        For Each vDimVal In vDimValArr
            Debug.Print "        " & vConfigName & " = " & vDimVal & " mm"
        Next vDimVal
    Next vConfigName

End Sub
```

## Get Dimension Values in Drawing Example (C#)

This example shows how to get the values of the dimensions in a drawing.

```
//-----
// Preconditions:
// 1. Open a drawing document.
// 2. Open the Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetDimensionValues_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);
            View swView = default(View);
            DisplayDimension swDispDim = default(DisplayDimension);
            Dimension swDim = default(Dimension);
            Annotation swAnn = default(Annotation);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;

            Debug.Print("File = " + swModel.GetPathName());

            swView = (View)swDraw.GetFirstView();

            while ((swView != null))
            {
                Debug.Print(" View = " + swView.Name);

                swDispDim = swView.GetFirstDisplayDimension5();

                while ((swDispDim != null))
                {
                    swAnn = (Annotation)swDispDim.GetAnnotation();
                    swDim = (Dimension)swDispDim.GetDimension();

                    Debug.Print(" -----");
                    Debug.Print(" AnnName = " + swAnn.GetName());
                    Debug.Print(" DimFullName = " + swDim.FullName);
                    Debug.Print(" DimName = " + swDim.Name);
                    Debug.Print(" swDimensionParamType_e type = " + swDim.GetType());
                    Debug.Print(" DrivenState = " + swDim.DrivenState);
                    Debug.Print(" ReadOnly = " + swDim.ReadOnly);
                    Debug.Print(" Value = " + swDim.GetSystemValue2(""));
                    Debug.Print("");
                    Debug.Print(" ArrowSide = " + swDispDim.ArrowSide);
                    Debug.Print(" TextAll = " +
                        swDispDim.GetText((int)swDimensionTextParts_e.swDimensionTextAll));
                    Debug.Print(" TextPrefix = " +
                        swDispDim.GetText((int)swDimensionTextParts_e.swDimensionTextPrefix));
                    Debug.Print(" TextSuffix = " +
                        swDispDim.GetText((int)swDimensionTextParts_e.swDimensionTextSuffix));
                    Debug.Print(" CalloutAbove = " +
                        swDispDim.GetText((int)swDimensionTextParts_e.swDimensionTextCalloutAbove));
                    Debug.Print(" CalloutBelow = " +
                        swDispDim.GetText((int)swDimensionTextParts_e.swDimensionTextCalloutBelow));

                    swDispDim = (DisplayDimension)swDispDim.GetNext3();

                }

                swView = (View)swView.GetNextView();

            }

        }

        public SldWorks swApp;
    }
}
```

## Get Dimension Values in Drawing Example (VBA)

This example shows how to get the values of the dimensions in a drawing.

```
'-----
' Preconditions:
' 1. Open a drawing document.
' 2. Open the Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----
```

Option Explicit

Sub main()

```
Dim swApp                As SldWorks.SldWorks
Dim swModel              As SldWorks.ModelDoc2
Dim swDraw               As SldWorks.DrawingDoc
Dim swView               As SldWorks.View
Dim swDispDim            As SldWorks.DisplayDimension
Dim swDim                As SldWorks.Dimension
Dim swAnn                As SldWorks.Annotation
```

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swDraw = swModel
```

```
Debug.Print "File = " & swModel.GetPathName
```

```
Set swView = swDraw.GetFirstView
```

```
Do While Not swView Is Nothing
    Debug.Print "    View = " & swView.Name
```

```
    Set swDispDim = swView.GetFirstDisplayDimension6
```

```
    Do While Not swDispDim Is Nothing
```

```
        Set swAnn = swDispDim.GetAnnotation
        Set swDim = swDispDim.GetDimension
```

```
        Debug.Print "    -----"
        Debug.Print "        AnnName = " & swAnn.GetName
        Debug.Print "        DimFullName           = " & swDim.FullName
        Debug.Print "        DimName              = " & swDim.Name
        Debug.Print "        swDimensionParamType_e type = " & swDim.GetType
        Debug.Print "        DrivenState          = " & swDim.DrivenState
        Debug.Print "        ReadOnly             = " & swDim.ReadOnly
        Debug.Print "        Value                 = " & swDim.GetSystemValue2("")
        Debug.Print ""
        Debug.Print "        ArrowSide            = " & swDispDim.ArrowSide
        Debug.Print "        TextAll              = " & swDispDim.GetText(swDimensionTextAll)
        Debug.Print "        TextPrefix          = " & swDispDim.GetText(swDimensionTextPrefix)
        Debug.Print "        TextSuffix          = " & swDispDim.GetText(swDimensionTextSuffix)
        Debug.Print "        CalloutAbove        = " & swDispDim.GetText(swDimensionTextCalloutAbove)
        Debug.Print "        CalloutBelow        = " & swDispDim.GetText(swDimensionTextCalloutBelow)
```

```
        Set swDispDim = swDispDim.GetNext3
```

```
    Loop
```

```
    Set swView = swView.GetNextView
```

```
Loop
```

End Sub

## Get Dimension Values in Drawing Example (VB.NET)

This example shows how to get the values of the dimensions in a drawing.

```

'-----
' Preconditions:
' 1. Open a drawing document.
' 2. Open the Immediate window.
'-----
' Postconditions: Inspect the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swDraw As DrawingDoc
        Dim swView As View
        Dim swDispDim As DisplayDimension
        Dim swDim As Dimension
        Dim swAnn As Annotation

        swModel = swApp.ActiveDoc
        swDraw = swModel

        Debug.Print("File = " & swModel.GetPathName)

        swView = swDraw.GetFirstView

        Do While Not swView Is Nothing
            Debug.Print(" View = " & swView.Name)

            swDispDim = swView.GetFirstDisplayDimension5

            Do While Not swDispDim Is Nothing
                swAnn = swDispDim.GetAnnotation
                swDim = swDispDim.GetDimension

                Debug.Print(" -----")
                Debug.Print(" AnnName = " & swAnn.GetName)
                Debug.Print(" DimFullName = " & swDim.FullName)
                Debug.Print(" DimName = " & swDim.Name)
                Debug.Print(" swDimensionParamType_e type = " & swDim.GetType)
                Debug.Print(" DrivenState = " & swDim.DrivenState)
                Debug.Print(" ReadOnly = " & swDim.ReadOnly)
                Debug.Print(" Value = " & swDim.GetSystemValue2(""))
                Debug.Print("")
                Debug.Print(" ArrowSide = " & swDispDim.ArrowSide)
                Debug.Print(" TextAll = " & swDispDim.GetText(swDimensionTextParts_e.swDimensionTextAll))
                Debug.Print(" TextPrefix = " &
                    swDispDim.GetText(swDimensionTextParts_e.swDimensionTextPrefix))
                Debug.Print(" TextSuffix = " &
                    swDispDim.GetText(swDimensionTextParts_e.swDimensionTextSuffix))
                Debug.Print(" CalloutAbove = " &
                    swDispDim.GetText(swDimensionTextParts_e.swDimensionTextCalloutAbove))
                Debug.Print(" CalloutBelow = " &
                    swDispDim.GetText(swDimensionTextParts_e.swDimensionTextCalloutBelow))

                swDispDim = swDispDim.GetNext3

            Loop

            swView = swView.GetNextView

        Loop

    End Sub

    Public swApp As SldWorks

End Class

```

## Get DimXpert Display Dimensions and Feature Example (C#)

This example shows how to find out if an annotation is a DimXpert display dimension, and, if so, how to get its DimXpert feature.

```
//-----
// Preconditions:
// 1. Add SolidWorks.Interop.swdimxpert.dll as a reference
//    (in the Project Explorer, right-click References, click Add Reference,
//    click Browse, and navigate to install_dir\api\redist).
// 2. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.
// 3. Click View > Toolbars > DimXpert.
// 4. Click the Auto Dimension Scheme button on the DimXpert toolbar.
// 5. Verify that Chamfer and Simple hole are selected in Feature Filters
//    in the Auto Dimension Scheme PropertyManager page and click OK.
// 6. Open the Immediate window.
//
// Postconditions:
// 1. Gets the DimXpert display dimensions in the model.
// 2. Examine the Immediate window.
//
// NOTE: Because this part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdimxpert;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swPart = default(ModelDoc2);
            Annotation swAnnotation = default(Annotation);
            DisplayDimension swDisplayDimension = default(DisplayDimension);

            swPart = (ModelDoc2)swApp.ActiveDoc;
            swAnnotation = (Annotation)swPart.GetFirstAnnotation2();
            while ((swAnnotation != null))
            {
                Debug.Print(" ");
                Debug.Print("Annotation name = " + swAnnotation.GetName());
                Debug.Print("Annotation DimXpert name = " + swAnnotation.GetDimXpertName());
                int AnnotationType = (int)swDimensionType_e.swDimensionTypeUnknown;
                AnnotationType = swAnnotation.GetType();
                if (AnnotationType == (int)swAnnotationType_e.swDisplayDimension)
                {
                    Debug.Print(" Is a display dimension? True");

                    swDisplayDimension = (DisplayDimension)swAnnotation.GetSpecificAnnotation();
                    switch ((swDisplayDimension.Type2))
                    {
                        case (int)swDimensionType_e.swOrdinateDimension:
                            Debug.Print(" Display dimension type = base ordinate and its subordinates");
                            break;
                        case (int)swDimensionType_e.swLinearDimension:
                            Debug.Print(" Display dimension type = linear");
                            break;
                        case (int)swDimensionType_e.swAngularDimension:
                            Debug.Print(" Display dimension type = angular");
                            break;
                        case (int)swDimensionType_e.swArcLengthDimension:
                            Debug.Print(" Display dimension type = arc length");
                            break;
                        case (int)swDimensionType_e.swRadialDimension:
                            Debug.Print(" Display dimension type = radial");
                            break;
                        case (int)swDimensionType_e.swDiameterDimension:
                            Debug.Print(" Display dimension type = diameter");
                            break;
                        case (int)swDimensionType_e.swHorOrdinateDimension:
                            Debug.Print(" Display dimension type = horizontal ordinate");
                            break;
                        case (int)swDimensionType_e.swVertOrdinateDimension:
                            Debug.Print(" Display dimension type = vertical ordinate");
                            break;
                        case (int)swDimensionType_e.swZAxisDimension:
                            Debug.Print(" Display dimension type = z-axis");
                            break;
                        case (int)swDimensionType_e.swChamferDimension:
                            Debug.Print(" Display dimension type = chamfer dimension");
                            break;
                        case (int)swDimensionType_e.swHorLinearDimension:
                    }
                }
            }
        }
    }
}
```



```

        Debug.Print(" Display dimension type = horizontal linear");
        break;
    case (int)swDimensionType_e.swVertLinearDimension:
        Debug.Print(" Display dimension type = vertical linear");
        break;
    case (int)swDimensionType_e.swScalarDimension:
        Debug.Print(" Display dimension type = scalar");
        break;
    default:
        Debug.Print(" Display dimension type = unknown");
        break;
}
Debug.Print(" Is a DimXpert display dimension? " + (swDisplayDimension.IsDimXpert() == false ? "False" :
"True"));
if (swAnnotation.IsDimXpert())
{
    DimXpertFeature DimXpertFeat = default(DimXpertFeature);
    string FeatName = null;

    DimXpertFeat = (DimXpertFeature)swAnnotation.GetDimXpertFeature();
    if ((DimXpertFeat != null))
    {
        FeatName = DimXpertFeat.Name;
        Debug.Print(" DimXpert feature name = " + FeatName);
        switch ((DimXpertFeat.Type))
        {
            case swDimXpertFeatureType_e.swDimXpertFeature_Plane:
                Debug.Print(" DimXpert feature type = plane");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_Cylinder:
                Debug.Print(" DimXpert feature type = cylinder");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_Cone:
                Debug.Print(" DimXpert feature type = cone");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_Extrude:
                Debug.Print(" DimXpert feature type = extrude");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_Fillet:
                Debug.Print(" DimXpert feature type = fillet");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_Chamfer:
                Debug.Print(" DimXpert feature type = chamfer");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole:
                Debug.Print(" DimXpert feature type = compound hole");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth:
                Debug.Print(" DimXpert feature type = compound width");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch:
                Debug.Print(" DimXpert feature type = compound notch");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D:
                Debug.Print(" DimXpert feature type = compound closed-slot 3D");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint:
                Debug.Print(" DimXpert feature type = intersect point");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine:
                Debug.Print(" DimXpert feature type = intersect line");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle:
                Debug.Print(" DimXpert feature type = intersect circle");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane:
                Debug.Print(" DimXpert feature type = intersect plane");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_Pattern:
                Debug.Print(" DimXpert feature type = pattern");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_Sphere:
                Debug.Print(" DimXpert feature type = sphere");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane:
                Debug.Print(" DimXpert feature type = best-fit plane");
                break;
            case swDimXpertFeatureType_e.swDimXpertFeature_Surface:
                Debug.Print(" DimXpert feature type = surface");
                break;
            default:
                Debug.Print(" DimXpert feature type = unknown");
                break;
        }
    }
}
else
{
    Debug.Print(" Not a display dimension.");
}
swAnnotation = (Annotation)swAnnotation.GetNext3();
}
}
/// <summary>

```

```
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}
```

## Get DimXpert Display Dimensions and Feature Example (VBA)

This example shows how to find out if an annotation is a DimXpert display dimension, and, if so, how to get its DimXpert feature.

```

'-----
' Preconditions:
' 1. Add a reference to the SOLIDWORKS version DimXpert type library.
' 2. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.
' 3. Click View > Toolbars > DimXpert.
' 4. Click the Auto Dimension Scheme button on the DimXpert toolbar.
' 5. Verify that Chamfer and Simple hole are selected in Feature Filters
'     in the Auto Dimension Scheme PropertyManager page and click OK.
' 6. Open the Immediate window.
'
' Postconditions:
' 1. Gets the DimXpert display dimensions in the model.
' 2. Examine the Immediate window.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swPart As SldWorks.ModelDoc2
Dim swAnnotation As SldWorks.Annotation
Dim swDisplayDimension As SldWorks.DisplayDimension

Sub main()

    Set swApp = Application.SldWorks
    Set swPart = swApp.ActiveDoc
    Set swAnnotation = swPart.GetFirstAnnotation2
    Do While (Not (swAnnotation Is Nothing))
        Debug.Print " "
        Debug.Print "Annotation name = " & swAnnotation.GetName
        Debug.Print "Annotation DimXpert name = " & swAnnotation.GetDimXpertName
        If swAnnotationType_e.swDisplayDimension = swAnnotation.GetType Then
            Debug.Print " Is a display dimension? True"
            Set swDisplayDimension = swAnnotation.GetSpecificAnnotation
            Select Case (swDisplayDimension.Type2)
                Case swDimensionType_e.swOrdinateDimension
                    Debug.Print " Display dimension type = base ordinate and its subordinates"
                Case swDimensionType_e.swLinearDimension
                    Debug.Print " Display dimension type = linear"
                Case swDimensionType_e.swAngularDimension
                    Debug.Print " Display dimension type = angular"
                Case swDimensionType_e.swArcLengthDimension
                    Debug.Print " Display dimension type = arc length"
                Case swDimensionType_e.swRadialDimension
                    Debug.Print " Display dimension type = radial"
                Case swDimensionType_e.swDiameterDimension
                    Debug.Print " Display dimension type = diameter"
                Case swDimensionType_e.swHorOrdinateDimension
                    Debug.Print " Display dimension type = horizontal ordinate"
                Case swDimensionType_e.swVertOrdinateDimension
                    Debug.Print " Display dimension type = vertical ordinate"
                Case swDimensionType_e.swZAxisDimension
                    Debug.Print " Display dimension type = z-axis"
                Case swDimensionType_e.swChamferDimension
                    Debug.Print " Display dimension type = chamfer dimension"
                Case swDimensionType_e.swHorLinearDimension
                    Debug.Print " Display dimension type = horizontal linear"
                Case swDimensionType_e.swVertLinearDimension
                    Debug.Print " Display dimension type = vertical linear"
                Case swDimensionType_e.swScalarDimension
                    Debug.Print " Display dimension type = scalar"
                Case Else
                    Debug.Print " Display dimension type = unknown"
            End Select

            Debug.Print " Is a DimXpert display dimension? " & IIf(swDisplayDimension.IsDimXpert = False, "False", "True")

            If swAnnotation.IsDimXpert Then
                Dim DimXpertFeat As SwDimXpert.DimXpertFeature
                Dim FeatName As String

                Set DimXpertFeat = swAnnotation.GetDimXpertFeature()
                If Not DimXpertFeat Is Nothing Then
                    FeatName = DimXpertFeat.Name
                    Debug.Print " DimXpert feature name = " & FeatName

                    Select Case (DimXpertFeat.Type)
                        Case swDimXpertFeatureType_e.swDimXpertFeature_Plane
                            Debug.Print " DimXpert feature type = plane"
                        Case swDimXpertFeatureType_e.swDimXpertFeature_Cylinder
                            Debug.Print " DimXpert feature type = cylinder"
                        Case swDimXpertFeatureType_e.swDimXpertFeature_Cone
                            Debug.Print " DimXpert feature type = cone"
                        Case swDimXpertFeatureType_e.swDimXpertFeature_Extrude
                            Debug.Print " DimXpert feature type =extrude"
                        Case swDimXpertFeatureType_e.swDimXpertFeature_Fillet
                            Debug.Print " DimXpert feature type = fillet"
                        Case swDimXpertFeatureType_e.swDimXpertFeature_Chamfer
                            Debug.Print " DimXpert feature type = chamfer"
                        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole
                            Debug.Print " DimXpert feature type = compound hole"
                    End Select
                End If
            End If
        End While
    End Sub

```

```

        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth
            Debug.Print " DimXpert feature type = compound width"
        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch
            Debug.Print " DimXpert feature type = compound notch"
        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D
            Debug.Print " DimXpert feature type = compound closed-slot 3D"
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint
            Debug.Print " DimXpert feature type = intersect point"
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine
            Debug.Print " DimXpert feature type = intersect line"
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle
            Debug.Print " DimXpert feature type = intersect circle"
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane
            Debug.Print " DimXpert feature type = intersect plane"
        Case swDimXpertFeatureType_e.swDimXpertFeature_Pattern
            Debug.Print " DimXpert feature type = pattern"
        Case swDimXpertFeatureType_e.swDimXpertFeature_Sphere
            Debug.Print " DimXpert feature type = sphere"
        Case swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane
            Debug.Print " DimXpert feature type = best-fit plane"
        Case swDimXpertFeatureType_e.swDimXpertFeature_Surface
            Debug.Print " DimXpert feature type = surface"
        Case Else
            Debug.Print " DimXpert feature type = unknown"
        End Select
    End If
End If
Else
    Debug.Print " Not a display dimension."
End If
Set swAnnotation = swAnnotation.GetNext3
Loop
End Sub

```

## Get DimXpert Display Dimensions and Feature Example (VB.NET)

This example shows how to find out if an annotation is a DimXpert display dimension, and, if so, how to get its DimXpert feature.

```

'-----
' Preconditions:
' 1. Add SolidWorks.Interop.swdimxpert.dll as a reference
'    (in the Project Explorer, right-click References, click Add Reference,
'    click Browse, and navigate to install_dir\api\redist).
' 2. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.
' 3. Click View > Toolbars > DimXpert.
' 4. Click the Auto Dimension Scheme button on the DimXpert toolbar.
' 5. Verify that Chamfer and Simple hole are selected in Feature Filters
'    in the Auto Dimension Scheme PropertyManager page and click OK.
' 6. Open the Immediate window.
'
' Postconditions:
' 1. Gets the DimXpert display dimensions in the model.
' 2. Examine the Immediate window.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdimxpert
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swPart As ModelDoc2
        Dim swAnnotation As Annotation
        Dim swDisplayDimension As DisplayDimension

        swPart = swApp.ActiveDoc
        swAnnotation = swPart.GetFirstAnnotation2
        Do While (Not (swAnnotation Is Nothing))
            Debug.Print(" ")
            Debug.Print("Annotation name = " & swAnnotation.GetName)
            Debug.Print("Annotation DimXpert name = " & swAnnotation.GetDimXpertName)
            If swAnnotationType_e.swDisplayDimension = swAnnotation.GetType Then
                Debug.Print(" Is a display dimension? True")
                swDisplayDimension = swAnnotation.GetSpecificAnnotation
                Select Case (swDisplayDimension.Type2)
                    Case swDimensionType_e.swOrdinateDimension
                        Debug.Print(" Display dimension type = base ordinate and its subordinates")
                    Case swDimensionType_e.swLinearDimension
                        Debug.Print(" Display dimension type = linear")
                    Case swDimensionType_e.swAngularDimension
                        Debug.Print(" Display dimension type = angular")
                    Case swDimensionType_e.swArcLengthDimension
                        Debug.Print(" Display dimension type = arc length")
                    Case swDimensionType_e.swRadialDimension
                        Debug.Print(" Display dimension type = radial")
                    Case swDimensionType_e.swDiameterDimension
                        Debug.Print(" Display dimension type = diameter")
                    Case swDimensionType_e.swHorOrdinateDimension
                        Debug.Print(" Display dimension type = horizontal ordinate")
                    Case swDimensionType_e.swVertOrdinateDimension
                        Debug.Print(" Display dimension type = vertical ordinate")
                    Case swDimensionType_e.swZAxisDimension
                        Debug.Print(" Display dimension type = z-axis")
                    Case swDimensionType_e.swChamferDimension
                        Debug.Print(" Display dimension type = chamfer dimension")
                    Case swDimensionType_e.swHorLinearDimension
                        Debug.Print(" Display dimension type = horizontal linear")
                    Case swDimensionType_e.swVertLinearDimension
                        Debug.Print(" Display dimension type = vertical linear")
                    Case swDimensionType_e.swScalarDimension
                        Debug.Print(" Display dimension type = scalar")
                    Case Else
                        Debug.Print(" Display dimension type = unknown")
                End Select
                Debug.Print(" Is a DimXpert display dimension? " & IIf(swDisplayDimension.IsDimXpert = False, "False", "True"))
                If swAnnotation.IsDimXpert Then
                    Dim DimXpertFeat As DimXpertFeature
                    Dim FeatName As String
                    DimXpertFeat = swAnnotation.GetDimXpertFeature()
                    If Not DimXpertFeat Is Nothing Then
                        FeatName = DimXpertFeat.Name
                        Debug.Print(" DimXpert feature name = " & FeatName)
                        Select Case (DimXpertFeat.Type)
                            Case swDimXpertFeatureType_e.swDimXpertFeature_Plane
                                Debug.Print(" DimXpert feature type = plane")
                        End Select
                    End If
                End If
            End If
            swAnnotation = swAnnotation.NextAnnotation
        Loop
    End Sub
End Class

```

```

        Case swDimXpertFeatureType_e.swDimXpertFeature_Cylinder
            Debug.Print(" DimXpert feature type = cylinder")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Cone
            Debug.Print(" DimXpert feature type = cone")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Extrude
            Debug.Print(" DimXpert feature type = extrude")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Fillet
            Debug.Print(" DimXpert feature type = fillet")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Chamfer
            Debug.Print(" DimXpert feature type = chamfer")
        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole
            Debug.Print(" DimXpert feature type = compound hole")
        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth
            Debug.Print(" DimXpert feature type = compound width")
        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch
            Debug.Print(" DimXpert feature type = compound notch")
        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D
            Debug.Print(" DimXpert feature type = compound closed-slot 3D")
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint
            Debug.Print(" DimXpert feature type = intersect point")
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine
            Debug.Print(" DimXpert feature type = intersect line")
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle
            Debug.Print(" DimXpert feature type = intersect circle")
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane
            Debug.Print(" DimXpert feature type = intersect plane")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Pattern
            Debug.Print(" DimXpert feature type = pattern")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Sphere
            Debug.Print(" DimXpert feature type = sphere")
        Case swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane
            Debug.Print(" DimXpert feature type = best-fit plane")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Surface
            Debug.Print(" DimXpert feature type = surface")
        Case Else
            Debug.Print(" DimXpert feature type = unknown")
        End Select
    End If
End If
Else
    Debug.Print(" Not a display dimension.")
End If
swAnnotation = swAnnotation.GetNext3
Loop
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class